

PKI / TLS / Service Identity — Operator Manual

What this technology does

PKI, TLS, mTLS, and workload identities define who a service is allowed to talk to and what identity is presented across the transport boundary.

Network reachability is not authorization. A TCP connection that succeeds while certificate validation, mTLS, or application ACLs are bypassed is not a healthy recovery.

Current trust purposes

The deployment separates certificate purposes where operationally practical:

```
public/service HTTPS identity
Gateway MQTT CA and per-gateway client identity
Gateway evidence upload PKI
PostgreSQL / PgBouncer trust
Valkey service trust
OpenBao internal KMS trust
SeaweedFS evidence-object-store internal trust
Fabric Gateway server trust + dedicated Fabric application identity
```

The current etcd cluster remains a documented private-HTTP POC exception. Do not copy that exception to other services.

Important current paths

Common protected locations include:

```
/etc/lorawan-pki/postgres/
/etc/lorawan-pki/pgbouncer/
/etc/lorawan-pki/mqtt/
/etc/lorawan-pki/valkey/
/etc/lorawan-pki/openbao/
/etc/lorawan-pki/gateway-evidence/
/etc/lorawan-pki/evidence-objectstore/
/etc/lorawan-pki/fabric/
```

The exact consumer and mode vary by service. Never loosen a private key to world-readable just because a container cannot open it.

Security hard stops

Never solve a TLS problem by permanently using:

```
curl -k
tls_skip_verify
verify none
sslmode=require when verify-full is expected
hostname verification disabled
```

```
shared administrator identity for an application worker
copying one gateway/node private key to another
printing private keys, passwords, SecretIDs, tokens, or unrestricted DSNs
```

Fix the trust chain, SAN/name, time, permissions, client identity, or ACL at the layer that actually failed.

Step 1 — Inventory certificates without printing private keys

Run on the affected host:

```
sudo find /etc/lorawan-pki -type f \(\ -name '*.crt' -o -name '*.pem' \) -printf '%M %u:%g %p\n'
2>/dev/null | sort
```

Then inspect only the certificate required by the failing service:

```
openssl x509 -in /path/to/certificate.crt -noout -subject -issuer -serial -dates -fingerprint -sha256 -
ext subjectAltName -ext extendedKeyUsage
```

PASS means: issuer, SAN, key usage, and validity window match the intended service/client role.

Step 2 — Check clock before diagnosing certificate dates

```
date -u
timedatectl status
```

A host clock outside the certificate validity window can look like a PKI failure. Correct time synchronization rather than reissuing good certificates blindly.

Step 3 — Verify hostname/SAN against the name clients use

Examples of commissioned logical names include:

```
mqtt.internal.lorawan.com
pgbouncer.internal.lorawan.com
openbao-kms.internal.lorawan.com
evidence-objects.internal.lorawan.com
peer1.hrc.local
```

Inspect SANs:

```
openssl x509 -in /path/to/server.crt -noout -ext subjectAltName
```

The client-visible name must be present. Do not “fix” a mismatch by switching the client to an unrelated IP name if that bypasses the commissioned service identity.

Step 4 — Verify a server TLS endpoint

Generic pattern:

```
openssl s_client -connect '<HOST>:<PORT>' -servername '<EXPECTED_TLS_NAME>' -verify_hostname
'<EXPECTED_TLS_NAME>' -verify_return_error -CAfile '/path/to/ca.crt' </dev/null
```

Require:

```
Verify return code: 0 (ok)
```

This proves server-auth TLS only. It does not prove application authorization or client mTLS unless a client identity is also presented.

Step 5 — Verify certificate/private-key pairing without revealing the key

```
CERT_PUB="$(  
  openssl x509 -in /path/to/client.crt -pubkey -noout |  
  openssl pkey -pubin -outform DER |  
  sha256sum | awk '{print $1}'  
)"  
  
KEY_PUB="$(  
  openssl pkey -in /path/to/client.key -pubout -outform DER |  
  sha256sum | awk '{print $1}'  
)"  
  
printf 'certificate_public_key_sha256=%s\n' "$CERT_PUB"  
printf 'private_key_public_key_sha256=%s\n' "$KEY_PUB"  
test "$CERT_PUB" = "$KEY_PUB"
```

PASS means: the public keys match. The private key body is never printed.

Step 6 — Check ownership and mode

```
sudo stat -c '%U:%G %a %n' /path/to/ca.crt /path/to/client.crt /path/to/client.key
```

For evidence/Fabric containers, protected private keys are normally mounted for numeric runtime GID 65532 with restrictive read-only permissions; public CAs/certificates must be readable by the runtime but not writable by untrusted users.

Use the owning component's preflight as the authority for exact required mode.

Step 7 — Verify the client-auth boundary separately

mTLS failures have two halves:

```
client verifies server  
server verifies client
```

For gateway MQTT, evidence upload, and collector broker identities, also verify:

- the client certificate chains to the expected client CA;
- Extended Key Usage permits client authentication;
- certificate identity maps to the intended workload or Gateway EUI;
- broker/API ACL grants only the intended topic/resource;
- an unrelated identity is rejected.

A valid certificate with an over-broad ACL is still a security defect.

Step 8 — Preserve service-identity separation

Current examples:

Gateway MQTT identity	-> gateway publish permissions only
Evidence-upload identity	-> one Gateway EUI upload authority
Collector identities	-> read as923/gateway/+/event/# only
Node-RED identity	-> application event subscription only
Verifier identity	-> verifier DB/object-read authority; no signing
Fabric adapter identity	-> dedicated OpenBao/Fabric authority
Grafana identity	-> read-only telemetry/evidence DB

Do not reuse a powerful identity across roles merely to reduce certificate count.

Step 9 — Check certificate expiry

For one certificate:

```
openssl x509 -in /path/to/certificate.crt -noout -enddate
openssl x509 -in /path/to/certificate.crt -checkend $((30*24*60*60)) -noout && echo 'EXPIRY_30D=PASS' ||
echo 'EXPIRY_30D=RENEW_REQUIRED'
```

Keep certificate-expiry monitoring active. Do not wait for production TLS failure to discover expiration.

Safe certificate rotation procedure

1. Identify every consumer of the certificate/CA/identity.
2. Prove the existing path is healthy before changing it.
3. Issue the replacement with the exact required SAN/EKU/purpose.
4. Verify the new cert/key pair offline.
5. Install it with correct owner/mode beside the current material.
6. If a CA changes, establish an overlap/trust-roll procedure where supported.
7. Rotate one service/member at a time.
8. Validate TLS/mTLS and application authorization.
9. Verify one representative real operation.
10. Revoke/remove the old identity only after every intended consumer has moved.

Do not rotate all HA members at once if one-at-a-time overlap is possible.

CA rotation rule

A CA change is more disruptive than a leaf-certificate renewal because both trusters and presenters may need an overlap period.

Before rotating a CA, list every issued service/client identity and trust-store mount, identify clients that cannot trust old+new simultaneously, define rollback, and preserve revocation/incident records.

Never replace a root CA on one side only and then disable verification to restore service.

Fabric-specific identity rule

The LoRaWAN Fabric adapter uses the HRC Gateway TLS root and a dedicated least-privilege client identity. It must not use HRC administrator material or unrelated benchmark identities.

The activation preflight verifies the Fabric TLS root fingerprint, `peer1.hrc.local` hostname, client certificate/private-key pairing, least-privilege identity class, and exact HRC Task 37 channel/chaincode/function contract.

Use the Fabric Adapter manual for that governed gate.

OpenBao-specific identity rule

OpenBao server TLS is separate from the adapter's AppRole authentication. A passing TLS handshake proves the KMS endpoint identity; it does not prove RoleID/SecretID validity or Transit policy.

Do not broaden TLS trust or Transit policy to compensate for an expired/invalid AppRole SecretID.

Gateway/evidence identity rule

Gateway MQTT and gateway evidence upload are separate trust purposes. Both must map unambiguously to the intended gateway, but one identity should not automatically grant the other's permissions.

For Gateway-01, certificate/ACL changes must preserve the commissioned Gateway EUI and plain AS923 data path.

Troubleshooting map

Symptom	First check
certificate expired/not-yet-valid	host clock + certificate dates
hostname mismatch	client name vs SAN; issue correct cert/name
unknown CA	correct trust chain/mounted CA
private key unreadable	owner/GID/mode/runtime UID
certificate/key mismatch	public-key hash comparison
server TLS passes, app says unauthorized	client identity/application ACL
mTLS client rejected	client chain/EKU/identity mapping
one HA node fails only	node-specific cert/SAN/path drift
rotating cert breaks peer	overlap/trust-store sequencing
Fabric TLS passes but submit fails	application identity/contract/Fabric layer

Completion checklist

- certificate chain and SAN match the name clients use;
- certificate validity and host time are correct;
- private keys have restrictive ownership/mode;

- certificate/private-key pairs match;
- TLS hostname verification is enabled;
- mTLS identities map to the intended workload/resource;
- least-privilege ACLs remain intact;
- no shared administrator identity has replaced a service identity;
- expiry monitoring/rotation procedure is known;
- representative authenticated operations pass after any rotation.

Detailed references

- [../server/cloud-production/04-host-hardening-dns-pki-and-secrets.md](#)
- [../server/cloud-production/08-mqtt-and-valkey.md](#)
- [../..evidence-services/cloud/deploy/README.md](#)
- [../server/integrations/hyperledger-fabric/02-fabric-network-handoff.md](#)

Supporting runbook: 04-host-hardening-dns-pki-and-secrets.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

4. Host Hardening, DNS, PKI, and Secrets

For the **step-by-step execution record on newly provisioned hosts**, use [04a-host-security-hardening-execution-runbook.md](#). That runbook records what was actually executed, why, the verification result, deviations from the supplied Ubuntu hardening document, and the explicitly excluded yellow-highlighted web-server section.

Current status: the host-security baseline is validated, and the major production PKI/secret boundaries for gateway MQTT, Evidence services, OpenBao, PostgreSQL-facing workloads, and the HRC Fabric adapter have since been commissioned in their component runbooks. Treat generic or future examples in this file as design guidance only; the component-specific current checkpoints and protected host configuration remain authoritative for actual listeners, certificate identities, and secret locations.

4.1 Baseline principles

- Use a supported LTS operating-system image and keep the exact cloud-image identifier for rebuild and rollback. This POC pins **Ubuntu Server 24.04 LTS x64 (Noble)** using the DigitalOcean ubuntu-24-04-x64 image on ha-01/02/03. Do not record only "Ubuntu LTS"; the exact release matters for package repositories and repeatable rebuilds.
- Use named operator accounts, SSH keys, and least privilege.
- Keep database, etcd, MQTT, and Patroni control ports on private interfaces.
- Use a dedicated service account for each daemon where practical.
- Store secrets outside Git and outside shell history.
- Treat transport security per service as a phase-specific control. The **current validated etcd checkpoint uses HTTP only on the private 10.104.0.0/20 east-west network**; do not claim etcd mTLS is deployed. If etcd transport TLS is added later, use a dedicated cluster CA and member/client identities. Gateway-facing MQTT and other future public/private services must be refined and validated when their phases become active.
- Apply updates through tested rolling procedures, not unattended major upgrades.

4.2 Initial host inspection

Run on every new host:

```
hostnamectl
uname -a
cat /etc/os-release
ip -br address
ip route
lsblk -f
```

```
findmnt
ss -lntup
systemctl --failed
journalctl -p warning -b --no-pager
```

Resolve unexpected listeners, failed units, duplicate addresses, or unmounted data volumes before installing the stack.

4.3 Operator access

Create a named operator and install an approved key through the cloud provisioning process. Confirm a second session works before changing SSH.

Back up and validate:

```
sudo cp -pn /etc/ssh/sshd_config /etc/ssh/sshd_config.before-lorawan-cloud
sudoedit /etc/ssh/sshd_config
sudo sshd -t
sudo systemctl reload ssh
```

Typical policy decisions:

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
AllowGroups <APPROVED_SSH_GROUP>
```

Preserve distribution include files and organization policy. Keep provider-console recovery available.

4.4 Time synchronization

Accurate time is required for TLS, logs, database recovery, and LoRaWAN event interpretation.

```
timedatectl status
timedatectl show -p NTPSynchronized --value
chronyc tracking 2>/dev/null || true
chronyc sources -v 2>/dev/null || true
```

Do not start etcd, Patroni, OpenBao, or certificate-sensitive commissioning while NTPSynchronized is no. Ubuntu may use a time-sync implementation other than chrony, which is why the timedatectl synchronization state is the required gate and the chronyc commands are supplemental when that client exists.

Alert on clock offset. Do not manually step time on a database/quorum node without understanding PostgreSQL, etcd lease, OpenBao, and certificate effects.

4.5 Kernel and resource limits

Review the limits below and change only values supported by measured demand. Keep each non-default value with the workload evidence that required it:

```
nofile limits for MQTT, HAProxy, PgBouncer, and PostgreSQL
vm.overcommit_memory policy
TCP listen and connection tracking capacity
filesystem mount options
transparent huge pages policy for PostgreSQL
```

```
swap policy
OOM behavior and systemd resource controls
```

Do not copy generic sysctl bundles. Apply one reviewed setting at a time and compare before/after behavior.

OpenBao and swap

A small host swap file may be useful as an emergency cushion for other processes, but **OpenBao must not be allowed to page secrets into ordinary unencrypted swap**. This is a security boundary, not a performance tweak.

Use the control that matches the OpenBao runtime selected later:

```
native/systemd OpenBao -> verify the service has MemorySwapMax=0
containerized OpenBao  -> configure the equivalent of --memory-swappiness=0
alternative             -> use an explicitly reviewed encrypted-swap design
```

Do not use swap to make an undersized OpenBao node appear healthy. Sustained host swapping still fails the 2-GiB sizing experiment.

4.6 Package and container runtime trust

- Use official distribution repositories or approved vendor repositories.
- Verify repository signing keys and fingerprints through an independent source.
- Pin container images by immutable digest.
- Keep the software bill of materials and vulnerability-scan result with each immutable image digest so the deployed artifact can be traced and replaced.
- Do not mount the container runtime socket into application containers.
- Do not run a container privileged unless the exact capability need is documented.

Install Docker Engine on the plain Ubuntu Server 24.04 LTS hosts

The DigitalOcean OS image is intentionally **not** a Docker Marketplace image, so install Docker Engine before any manual that runs `docker compose`. Docker's current Ubuntu instructions support Ubuntu Noble 24.04 LTS.

First prove that the host is the expected release. Run on **each** Droplet:

```
. /etc/os-release
printf 'ID=%s VERSION_ID=%s CODENAME=%s ARCH=%s\n' \
  "$ID" "$VERSION_ID" "${UBUNTU_CODENAME:-$VERSION_CODENAME}" "$(dpkg --print-architecture)"

test "$ID" = ubuntu
test "$VERSION_ID" = 24.04
test "${UBUNTU_CODENAME:-$VERSION_CODENAME}" = noble
test "$(dpkg --print-architecture)" = amd64
```

Stop here if any test fails. Do not adapt the repository command to a different Ubuntu release on the fly; either rebuild the mismatched Droplet or deliberately revise and revalidate the whole three-host baseline.

Before installing Docker, inspect the package names Docker currently documents as conflicts with Docker Engine from its official repository:

```
dpkg --get-selections \
  docker.io docker-compose docker-compose-v2 docker-doc docker-buildx \
```

```
podman-docker containerd runc \  
2>/dev/null || true
```

A fresh POC host should not contain an existing container workload. If any conflicting package is installed, first verify that `/var/lib/docker`, `/var/lib/containerd`, and the running process list do not contain an unknown workload. Only on a confirmed fresh host, remove the conflicts using the current Docker-documented package list:

Build the removal list first so an empty result does not turn into an ambiguous `apt remove` command:

```
CONFLICTING_DOCKER_PACKAGES="$(dpkg --get-selections \  
docker.io docker-compose docker-compose-v2 docker-doc docker-buildx \  
podman-docker containerd runc 2>/dev/null \  
| awk '$2 == "install" {print $1}')"\  
  
printf 'conflicting installed packages:\n%s\n' \  
"${CONFLICTING_DOCKER_PACKAGES:-<none>}"
```

If the list is non-empty **and this is confirmed to be a fresh host with no workload**, remove exactly that list:

```
if [ -n "$CONFLICTING_DOCKER_PACKAGES" ]; then\  
    sudo apt remove -y $CONFLICTING_DOCKER_PACKAGES\  
fi
```

An empty list is normal on a plain image. Do **not** delete `/var/lib/docker` or `/var/lib/containerd` as part of this procedure, and do not run the removal block on a host whose existing container state has not been identified.

Configure Docker's official **Ubuntu** repository:

```
sudo apt update\  
sudo apt install -y ca-certificates curl\  
sudo install -m 0755 -d /etc/apt/keyrings\  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \  
-o /etc/apt/keyrings/docker.asc\  
sudo chmod a+r /etc/apt/keyrings/docker.asc\  
  
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF\  
Types: deb\  
URIs: https://download.docker.com/linux/ubuntu\  
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")\  
Components: stable\  
Architectures: $(dpkg --print-architecture)\  
Signed-By: /etc/apt/keyrings/docker.asc\  
EOF\  
  
sudo apt update\  
apt list --all-versions docker-ce 2>/dev/null | head -n 20
```

Select and record the **approved Docker Engine version string** in the software worksheet. Do not copy a Docker version number from this manual because repository availability changes over time. Then install that reviewed engine version with Docker's current Ubuntu package set:

```
DOCKER_VERSION_STRING='<APPROVED_DOCKER_VERSION_STRING>'  
  
sudo apt install -y \  
docker-ce="$DOCKER_VERSION_STRING" \  
docker-ce-cli="$DOCKER_VERSION_STRING" \  
containerd.io \  
docker-buildx-plugin \  
docker-compose-plugin
```

Verify before deploying any application container:

```
sudo systemctl is-enabled docker
sudo systemctl is-active docker
sudo docker version
sudo docker compose version
dpkg-query -W \
    docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo docker info --format '{{.ServerVersion}} {{.LoggingDriver}}'
```

Record the Docker Engine, Compose plugin, containerd, package versions, and package source in [02-capacity-cost-and-ip-plan.md](#). Keep all three hosts on the same reviewed package baseline during initial commissioning.

Bound container logs before creating the stack

Current evidence gap: the live execution log proves Docker Engine 29.7.2, overlayfs, systemd cgroups, cgroup v2, and working docker compose use, but it does **not** contain captured output proving the current default Docker logging driver or that `/etc/docker/daemon.json` was changed to `local`. Treat the procedure below as required hardening to verify before the next container phase; do not retroactively mark it PASS.

Docker's default `json-file` driver does not rotate logs unless configured. On these 50-GiB POC disks, unbounded container logs are an avoidable failure mode.

For a new host, use Docker's rotating `local` logging driver as the default:

```
sudo install -d -m 0755 /etc/docker
```

If `/etc/docker/daemon.json` already exists, **stop and merge this setting with the existing reviewed JSON instead of overwriting it**. On a new empty host, create:

```
sudo tee /etc/docker/daemon.json >/dev/null <<'JSON'
{
  "log-driver": "local"
}
JSON

sudo systemctl restart docker
sudo docker info --format '{{.LoggingDriver}}'
```

Expected result:

```
local
```

This default applies to newly created containers. If containers existed before the change, recreate them only through their normal deployment procedure so they inherit the reviewed logging driver.

Docker and firewall boundary

Do not assume a host UFW rule alone protects ports published by Docker. Docker manages its own packet-filter rules. The target architecture requires a provider-side DigitalOcean Cloud Firewall, explicit private/loopback/anchor IP binds, and `ss -lntup` listener verification. **Current provider-firewall state is not verified by this operator and must not be claimed as active.** Until the provider owner confirms it, rely only on controls that are actually evidenced—service bind addresses, SSH hardening, Fail2ban, and listener inspection—and keep provider-firewall completion as an explicit outstanding control.

Spilo maintainers encourage users to build reviewed images from source because public image releases are not regular. Build in CI, scan, sign, publish to an approved registry, and pin the digest.

4.7 PKI hierarchy

The POC is small, but the certificate **roles and names must match the future deployment shape**. Reduce certificate count only when two functions genuinely share the same trust boundary; do not disable TLS or hostname verification to save setup time.

Use separate trust purposes where operationally practical:

```
Public CA
-> chirpstack.<DOMAIN>

Gateway MQTT CA
-> mqtt.<DOMAIN> broker server certificates
-> unique clientAuth certificate per gateway
-> separate ChirpStack and integration client certificates

Gateway Evidence CA or separately constrained PKI purpose
-> evidence.<DOMAIN> HTTPS server certificate
-> unique evidence-upload client identity per gateway
-> no MQTT publish authorization implied by this certificate

Internal etcd CA
-> unique peer/client certificate per etcd member
-> Patroni etcd client certificate

Internal PostgreSQL CA
-> unique server certificate per DB node
-> PgBouncer server certificates
-> optional application client certificates

Internal Valkey CA
-> unique Valkey/Sentinel certificate per node

Internal OpenBao CA
-> unique OpenBao server certificate per node
```

The gateway MQTT certificate Common Name equals the 16-hex Gateway ID when the broker maps certificate identity to ACL username. The evidence-upload identity must also map unambiguously to exactly one Gateway EUI, but it should use a separate key/purpose from MQTT when operationally practical. Reuse one key across MQTT and evidence upload only under an explicit reviewed PKI decision. Do not reuse one private key across gateways or nodes. Server certificates must include the exact DNS names and private IPs used by clients in Subject Alternative Names.

4.7A Certificate/SAN matrix for this POC

This matrix began as a future service-PKI target, but several rows are now commissioned. Use the names below as the current logical-name baseline for services already deployed; later execution records remain authoritative for exact certificate fingerprints, node SANs, and file paths. The current etcd cluster remains the explicit exception: it is validated with HTTP on the private east-west network, so its TLS row applies only if a later transport-hardening change is scheduled.

Service	Where	Name/SAN that clients verify	Why
ChirpStack HTTPS	HAProxy ha-01/02 anchor listeners	chirpstack.<DOMAIN>	Browser/API TLS through the Reserved IPv4 after ownership moves between app hosts
Mosquitto-1	ha-01	mqtt.internal.lorawan.com plus issued node SANs	Commissioned broker identity verified through HAProxy TLS passthrough

Service	Where	Name/SAN that clients verify	Why
Mosquitto-2	ha-02	mqtt.internal.lorawan.com plus issued node SANs	Backup broker presents the same commissioned service identity
PgBouncer	ha-01/02/03	pgbouncer.internal.lorawan.com, node private name/IP	Local database clients use one stable logical name
PostgreSQL	ha-01/02/03	postgres-ha.internal, node private name/IP	PgBouncer/HAProxy can verify any promoted primary
Valkey	ha-01/02/03	valkey.internal.lorawan.com, node private name/IP	HAProxy/ChirpStack verify any Sentinel-promoted primary
OpenBao	ha-01/02/03	openbao-kms.internal.<DOMAIN>, node private name/IP	Either active or standby can serve the stable KMS path
etcd - future TLS hardening only	ha-01/02/03	each member's private DNS/IP	If transport TLS is added later, peer/client mTLS must identify the actual member; current validated etcd is HTTP on 10.104.0.0/20

Client identities:

```
physical gateway MQTT client -> unique cert, CN = Gateway EUI
ChirpStack MQTT client(s)    -> workload identity, never a gateway identity
Node-RED MQTT client         -> read-only application-event identity
Fabric adapter-1/2           -> separate OpenBao workload identities where practical
```

Stop here. Do not continue if a client will connect using a name absent from the server certificate SAN. Fix the certificate or the service name; do not fall back to `sslmode=require`, disabled hostname checks, or `verify none` as the permanent solution.

4.8 Certificate file layout

Example:

```
/etc/lorawan-pki/
public/
etcd/
postgres/
pgbouncer/
mqtt/
valkey/
openbao/
gateway-ca/
gateway-evidence/  # when the evidence-ingest service is deployed
```

Recommended permissions:

```
sudo install -d -m 750 -o root -g <SERVICE_GROUP> /etc/lorawan-pki/<PURPOSE>
sudo install -m 640 -o root -g <SERVICE_GROUP> <CERT_FILE> /etc/lorawan-pki/<PURPOSE>/
sudo install -m 640 -o root -g <SERVICE_GROUP> <KEY_FILE> /etc/lorawan-pki/<PURPOSE>/
```

Some services require a different owner. Verify the effective service user and never loosen a private key to world-readable as a shortcut.

4.9 Secret inventory

At minimum, manage these independently:

- PostgreSQL superuser, replication, rewind, ChirpStack, backup, and monitoring credentials;
- PgBouncer authentication query or auth-file credential;
- etcd client and peer private keys;
- private MQTT broker administration, ChirpStack, Node-RED, and monitoring identities;
- unique Gateway OS MQTT client private keys and certificate lifecycle records;
- when gateway evidence is enabled, unique gateway evidence-upload client identities/private keys and lifecycle records;
- evidence-ingest service credential, protected evidence-store credential, and verifier database/object-store identities;
- ChirpStack token secret and API credentials;
- Valkey password or certificate bundle;
- object-storage access key scoped to the backup bucket **only when the later WAL/object-storage backup profile is enabled**; it is not required for the minimal HA POC;
- DigitalOcean API token used by automation, including the dedicated public-ingress Reserved-IP failover identity on ha-01/02;
- alerting/webhook credentials;
- OpenBao initialization/recovery material and unseal or reviewed auto-unseal material, stored independently from the running nodes;
- separate least-privilege OpenBao workload identities for Fabric adapter-1 and Fabric adapter-2;
- Fabric Gateway CA certificate, MSP ID, channel/chaincode metadata, and a protected client identity for each adapter worker where the Fabric team's policy supports separate identities;
- Fabric adapter database credentials limited to the telemetry/outbox permissions defined by the integration manual.

Keep gateway-evidence verifier credentials intentionally separate from OpenBao Transit sign authorization and the Fabric client private key. The verifier decides whether source lineage is valid; it must not also own the evidence-signing capability. Likewise, do not copy one Fabric client private key or one long-lived OpenBao adapter credential across both adapter hosts unless the external Fabric/OpenBao identity design explicitly requires and approves that sharing; separate workload identity gives clearer revocation and audit evidence.

For every secret, keep its purpose, consumers, protected storage reference, rotation or expiry trigger, and emergency revocation procedure. Do not place the secret value in the inventory.

4.10 Environment files

Use root-owned files outside the repository:

```
sudo install -d -m 750 /etc/lorawan-cloud
sudo install -m 600 /dev/null /etc/lorawan-cloud/<SERVICE>.env
sudoedit /etc/lorawan-cloud/<SERVICE>.env
```

Before using a rendered Compose configuration, remember that `docker compose config` may print expanded secret values. Prefer `docker compose config --quiet` for syntax validation, and redirect any necessary rendered output to a protected temporary file that is removed immediately.

4.11 DNS and certificate validation

```
getent ahosts <SERVICE_FQDN>
openssl s_client -connect <SERVICE_FQDN>:<PORT> -servername <SERVICE_FQDN> -showcerts </dev/null

# Also test evidence.<DOMAIN>:443 separately once that mTLS service exists.
```

Verify chain, hostname, expiry, key usage, and issuer. Automate expiry alerts with enough time for rollback.

4.12 Audit and logging hygiene

- Do not log AppKey, NwkKey, PostgreSQL passwords, private keys, MQTT passwords, API tokens, or complete connection strings.
- Redact authorization headers and database DSNs.
- Restrict journal and log-aggregation access.
- Define retention and legal requirements.
- Test that debug mode is off after commissioning.

4.13 Hardening acceptance

Current server-first security gaps — 2026-08-29

The original host baseline is accepted, but a later full-stack audit found service-security items that are **not** closed merely by the host-hardening PASS:

- certificate-expiry monitoring is required below, but no commissioned checker/timer is recorded;
- Fabric/KMS incident response depends on OpenBao audit evidence, but no commissioned OpenBao audit device is recorded;
- only the Spilo image has a detailed recorded vulnerability/hardening review; the other immutable service images still need scan/provenance coverage or an explicitly accepted gap;
- Docker log/resource/security options are documented consistently for Spilo/OpenBao, but Node-RED, Grafana, ChirpStack, and etcd require a read-only effective-runtime inventory before any hardening recreate is justified;
- etcd private HTTP remains an explicit POC transport-security exception, not production transport-security signoff;
- Historical commissioning began with UFW inactive, but the current hosts have since been hardened with UFW. Preserve the September 3 narrow Docker-bridge allowances required by ChirpStack (172.18.0.0/16 -> each node-local 10.104.0.x on 16379, 6432, 18883) and Node-RED (172.18.0.0/16 -> 10.104.0.8 on 18884, 6432). Re-audit the effective rules and Docker packet-filter interaction before any firewall rewrite; the provider Cloud Firewall state remains a separate externally managed boundary.

The earlier server-first sequence is complete: the gateway/security evidence runtime is commissioned, the OpenBao audit boundary is closed, and ULC-01 has a commissioned least-privilege Fabric writer. Do not repeat those commissioning steps merely because this older hardening guide describes them as future work. Current changes should preserve certificate-expiry monitoring, deterministic exact-byte validation, and container-security inventory; the open Fabric infrastructure boundary is ULC-02 HA fencing/ownership acceptance, while the September 17 research backlog is upstream outbox finalization/claim eligibility.

Complete only when:

- SSH password and root login policy is verified without lockout;

- public listeners match the approved matrix;
- every private key has a named owner and restrictive mode;
- deployed image digests, package versions, and sources can be identified;
- certificate expiry monitoring is active;
- secrets can be rotated without rebuilding the whole environment;
- logs contain no live secret during a sanitized test;
- when gateway evidence is deployed, one unauthorized/unknown gateway client is rejected by the evidence API, one authorized gateway identity maps only to its own Gateway EUI, and the verifier cannot use OpenBao/Fabric signing credentials.

Next validated build step: [05-etcd-cluster.md](#).

Supporting runbook: 08-mqtt-and-valkey.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

8. MQTT and Valkey HA on the Three-Node POC

Status: CORE SERVICE LAYER COMPLETE / VALIDATED. Mosquitto 2.0.18 is commissioned on `u1c-01` and `u1c-02` with TLS backend listeners on `:8884`, and the validated MQTT HAProxy TLS passthrough endpoint is `10.104.0.2:8883` using broker certificate identity `mqtt.internal.lorawan.com`; broker-backend failure and recovery have passed. Valkey 7.2.13 is commissioned TLS-only on all three nodes with authenticated replication, three TLS Sentinels (quorum 2), a least-privilege HAProxy health identity, dual writable-primary HAProxy endpoints `10.104.0.2:16379` and `10.104.0.4:16379`, and automatic promotion/rejoin testing. The most recent **Phase 8C.14 commissioning failover checkpoint** elected `u1c-02 (10.104.0.4)` as Valkey primary. Treat that as dated acceptance evidence, not a claim about today's elected primary; do not manually fail back solely to match documentation. ChirpStack workload MQTT authentication/ACLs and the second application-node MQTT routing boundary are intentionally deferred to Phase 9 before first ChirpStack start.

Execution-record authority: sections 8.1-8.17 preserve the original target design and are useful for rationale, but they contain placeholders and assumptions that were superseded during live commissioning. When those planning sections differ from the observed deployment, sections 8.18-8.21 and the live-state summary above are authoritative. In particular, the commissioned Valkey TLS identity is `valkey.internal.lorawan.com`, not `valkey-ha.internal.<DOMAIN>`, and the commissioned MQTT path is currently `10.104.0.2:8883 -> Mosquitto :8884`, not the uncommissioned `mqtt-ha.internal.<DOMAIN>:18883` design.

This manual deploys the two shared runtime dependencies used by ChirpStack:

```
MQTT transport      -> Mosquitto-1 preferred / Mosquitto-2 backup
fast shared state   -> Valkey primary + 2 replicas, managed by 3 Sentinels
```

The physical gateway's local Mosquitto queue remains separate on the Raspberry Pi. Do not confuse that edge buffer with the two cloud brokers.

8.1 Port plan and why the broker backend uses 8884

Use this exact separation:

```
PUBLIC MQTT SERVICE
mqtt.<DOMAIN>:8883
-> DigitalOcean Reserved IPv4
-> current owner: HAProxy on ha-01 OR ha-02 anchor :8883
-> Mosquitto backend TLS :8884

PRIVATE MQTT SERVICE
mqtt-ha.internal.<DOMAIN>:18883
-> local HAProxy on ha-01/ha-02/ha-03
-> same Mosquitto backend TLS :8884
```

8883 belongs to the HAProxy public frontend on the **anchor IP** of ha-01/02. The Mosquitto processes listen on private backend port 8884, so the broker and public HAProxy listener never contend for the same address/port.

Why: HAProxy and Mosquitto cannot both bind the same `<HOST_PRIVATE_IP>:8883`. Using 8884 internally removes that collision while preserving the normal public MQTT endpoint `mqtt.<DOMAIN>:8883`.

The cloud brokers do not need a plaintext 1883 listener. The only plaintext 1883 in this architecture is the Raspberry Pi gateway-local loopback broker at `127.0.0.1:1883`.

8.2 Preconditions

Before installing either service, record:

```
ha-01 private IP:
ha-02 private IP:
ha-03 private IP:
DOMAIN:
confirmed ChirpStack region/topic prefix:
Mosquitto image/version/digest:
Valkey image/version/digest:
MQTT CA path:
Valkey CA path:
Valkey shared application secret reference:
Sentinel administration secret reference:
```

Required certificates already issued by the PKI step:

```
Mosquitto-1 server certificate
  SAN: mqtt.<DOMAIN>
  SAN: mqtt-ha.internal.<DOMAIN>
  SAN: ha-01 private name/IP as required

Mosquitto-2 server certificate
  SAN: mqtt.<DOMAIN>
  SAN: mqtt-ha.internal.<DOMAIN>
  SAN: ha-02 private name/IP as required

Valkey-1/2/3 server certificates
  SAN: valkey-ha.internal.<DOMAIN>
  SAN: each node's own private name/IP
```

MQTT client identities required for the POC:

```
one unique certificate per physical gateway
  CN = its 16-hex Gateway EUI

ChirpStack workload certificate(s)
  not reused by gateways

Node-RED workload certificate
  read-only application-uplink permission
```

Stop here. Do not continue if certificate SANs, private IPs, region prefix, or image versions are still placeholders.

8.3 Prepare Mosquitto on ha-01 and ha-02

Run this section on **ha-01 and ha-02 only**.

Create directories:

```
sudo install -d -m 750 /etc/lorawan-cloud/mosquitto
sudo install -d -m 750 /etc/lorawan-pki/mqtt
sudo install -d -m 750 /srv/mosquitto/data
sudo install -m 640 /dev/null /etc/lorawan-cloud/mosquitto/acl
```

Install the node's broker certificate/key and the gateway/workload CA using the owner/group required by the pinned Mosquitto image or package. Never copy a CA private key to a broker.

Create `/etc/lorawan-cloud/mosquitto/mosquitto.conf` on each broker host. Replace `<THIS_HOST_PRIVATE_IP>` with that host's VPC address:

```
persistence true
persistence_location /mosquitto/data/
autosave_interval 60
log_dest stdout

listener 8884 <THIS_HOST_PRIVATE_IP>
protocol mqtt
allow_anonymous false

cafile /mosquitto/config/certs/ca.crt
certfile /mosquitto/config/certs/server.crt
keyfile /mosquitto/config/certs/server.key
require_certificate true
use_identity_as_username true
tls_version tlsv1.2

acl_file /mosquitto/config/acl
```

`require_certificate true` makes the TLS listener require a valid client certificate. `use_identity_as_username true` maps the client-certificate Common Name to the MQTT username used by the ACL.

Do not add a cloud listener on 1883.

8.4 Create the MQTT ACL

For each gateway, add only its own topics. Example shape:

```
user <GATEWAY_EUI>
topic write <REGION_PREFIX>/gateway/<GATEWAY_EUI>/event/#
topic write <REGION_PREFIX>/gateway/<GATEWAY_EUI>/state/#
topic read <REGION_PREFIX>/gateway/<GATEWAY_EUI>/command/#
```

For ChirpStack, give the reviewed workload identity the gateway-backend permissions it actually needs:

```
user <CHIRPSTACK_MQTT_IDENTITY>
topic read <REGION_PREFIX>/gateway/+/event/#
topic read <REGION_PREFIX>/gateway/+/state/#
topic write <REGION_PREFIX>/gateway/+/command/#
topic write application/+/device/+/event/#
topic read application/+/device/+/command/#
```

For Node-RED:

```
user <NODE_RED_MQTT_IDENTITY>
topic read application/+/device/+/event/up
```

If the pinned ChirpStack configuration uses separate gateway-backend and application-integration certificates, split the ChirpStack ACL accordingly rather than broadening one identity.

Copy the **same reviewed ACL policy** to Mosquitto-1 and Mosquitto-2. Keep certificate/private-key files node-specific.

8.5 Run each Mosquitto backend

A containerized host-network example is below. Use the exact paths/user documented by the pinned image:

```
services:
  mosquitto:
    image: <PINNED_MOSQUITTO_IMAGE>
    network_mode: host
    restart: unless-stopped
    volumes:
      - /etc/lorawan-cloud/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - /etc/lorawan-cloud/mosquitto/acl:/mosquitto/config/acl:ro
      - /etc/lorawan-pki/mqtt:/mosquitto/config/certs:ro
      - /srv/mosquitto/data:/mosquitto/data
```

Validate before start using the validation method supported by the pinned image, then:

```
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml config --quiet
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml up -d
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml ps
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml logs --since=5m --tail=100
sudo ss -lntp | grep ':8884'
```

Pass only when the listener is on the intended private address and no cloud :1883 listener exists.

8.6 Prove Mosquitto TLS and ACL before HAProxy

From an approved VPC host, use a staging client certificate:

```
mosquitto_pub \
  -h <MOSQUITTO_PRIVATE_IP> -p 8884 \
  --cafile <MQTT_CA> \
  --cert <STAGING_CLIENT_CERT> \
  --key <STAGING_CLIENT_KEY> \
  -t '<AUTHORIZED_TEST_TOPIC>' \
  -m 'mqtt-ha-preflight' -q 1
```

Then attempt one topic that the identity is **not** allowed to publish/read. The authorized operation must work and the cross-identity operation must be denied.

A successful TLS handshake alone is not enough; ACL direction must also be proven.

8.7 Add the HAProxy MQTT frontends

Use the **same preferred/backup ordering on every HAProxy host** so clients do not split across two unrelated brokers.

On ha-01 and ha-02, add both frontends:

```
frontend mqtt_public
  mode tcp
  bind <THIS_HOST_ANCHOR_IP>:8883
  default_backend mosquitto_active_standby
```

```

frontend mqtt_internal
    mode tcp
    bind <THIS_HOST_PRIVATE_IP>:18883
    default_backend mosquitto_active_standby

backend mosquitto_active_standby
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2
    server mosquitto-1 <HA01_PRIVATE_IP>:8884 check
    server mosquitto-2 <HA02_PRIVATE_IP>:8884 check backup

```

On ha-03, add **only** the private internal frontend plus the same backend:

```

frontend mqtt_internal
    mode tcp
    bind <HA03_PRIVATE_IP>:18883
    default_backend mosquitto_active_standby

backend mosquitto_active_standby
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2
    server mosquitto-1 <HA01_PRIVATE_IP>:8884 check
    server mosquitto-2 <HA02_PRIVATE_IP>:8884 check backup

```

The runtime HAProxy check is intentionally a Layer-4 availability check. End-to-end certificate and ACL correctness is proven separately with actual MQTT clients because the broker requires client certificates.

Validate and reload on each affected host:

```

sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
sudo systemctl reload haproxy
sudo ss -lntp | grep -E ':(8883|18883)\b'

```

Expected:

```

ha-01: 8883 + 18883
ha-02: 8883 + 18883
ha-03: 18883 only

```

8.8 MQTT service naming: target design versus commissioned state

The original design below used a local-per-host service name on :18883. That route was **not** commissioned during Phase 8B and must not be treated as live:

```

mqtt-ha.internal.<DOMAIN>:18883    # target only; not commissioned

```

The currently validated cloud MQTT path is:

```

client TLS hostname/SNI: mqtt.internal.lorawan.com
HAProxy frontend:       10.104.0.2:8883
Mosquitto backends:     10.104.0.2:8884 and 10.104.0.4:8884
TLS termination:        Mosquitto, not HAProxy

```

HAProxy forwards MQTT TLS bytes unchanged, so clients must verify the certificate presented by the selected Mosquitto backend. Phase 9 must not start ChirpStack until it closes two remaining items: a ChirpStack-specific MQTT authentication/ACL identity, and a routing design that does not make the second ChirpStack instance depend on a single u1c-01 HAProxy process. If a local-per-app-node MQTT frontend is commissioned later, its

hostname must match an actually issued broker certificate identity; do not reuse the obsolete `mqtt-ha.internal.<DOMAIN>` placeholder unless new certificates explicitly contain it.

8.9 MQTT failover rehearsal

Before moving on to ChirpStack HA:

1. verify Mosquitto-1 is the selected backend from all three HAProxy hosts;
2. connect one staging MQTT client through `mqtt-ha.internal.<DOMAIN>:18883`;
3. publish/receive one authorized test message;
4. stop **Mosquitto-1 only**;
5. record the stop timestamp;
6. wait for HAProxy to mark it unavailable;
7. reconnect the staging client without changing hostname, certificate, or port;
8. prove Mosquitto-2 serves the connection;
9. restart Mosquitto-1;
10. verify it is healthy and returns to preferred status;
11. confirm no broker is unintentionally listening on cloud 1883.

Later, repeat the same failure using the physical gateway and its local QoS 1 queue.

Do not call this broker-state replication. Mosquitto-1 and Mosquitto-2 do not share sessions or queues.

8.10 Prepare Valkey storage and certificates

Run on **all three hosts**:

```
sudo install -d -m 750 /etc/lorawan-cloud/valkey
sudo install -d -m 750 /etc/lorawan-pki/valkey
sudo install -d -m 700 /srv/valkey/data
sudo install -d -m 750 /srv/valkey/sentinel
```

Use one protected long random application/replication secret for this tiny POC and a separate Sentinel-administration secret. Keep them outside Git and shell history.

8.11 Configure the initial Valkey primary and replicas

Use `ha-01` as the **bootstrap primary only**. Sentinel may later promote either replica.

Common secure baseline, adapted per host:

```
bind <THIS_HOST_PRIVATE_IP>
protected-mode yes
port 0
tls-port 6379

tls-cert-file /run/valkey-tls/server.crt
tls-key-file /run/valkey-tls/server.key
tls-ca-cert-file /run/valkey-tls/ca.crt
tls-auth-clients no
tls-replication yes

requirepass <VALKEY_SECRET>
masterauth <VALKEY_SECRET>
```

```
appendonly yes
appendfsync everysec
maxmemory 128mb
maxmemory-policy noeviction
```

On ha-02 and ha-03, additionally configure:

```
replicaof <HA01_PRIVATE_IP> 6379
```

Do not configure replicaof on the bootstrap primary ha-01.

noeviction is deliberate for the POC: memory exhaustion should fail visibly instead of silently discarding state.

If the pinned Valkey version uses ACLs instead of the password-only baseline, use a named least-privilege application/replication user and configure Sentinel authentication consistently. Do not mix incompatible ACL/password schemes accidentally.

8.12 Configure Sentinel on all three hosts

Create a writable sentinel.conf per host. Sentinel rewrites this file as topology changes, so do not mount it read-only.

Baseline:

```
bind <THIS_HOST_PRIVATE_IP>
protected-mode yes
port 0
tls-port 26379

tls-cert-file /run/valkey-tls/server.crt
tls-key-file /run/valkey-tls/server.key
tls-ca-cert-file /run/valkey-tls/ca.crt
tls-auth-clients no
tls-replication yes

requirepass <SENTINEL_SECRET>

sentinel monitor lorawan-valkey <HA01_PRIVATE_IP> 6379 2
sentinel auth-pass lorawan-valkey <VALKEY_SECRET>
sentinel down-after-milliseconds lorawan-valkey 5000
sentinel failover-timeout lorawan-valkey 60000
sentinel parallel-syncs lorawan-valkey 1
```

The 2 in sentinel monitor is the failure-detection quorum. Actual failover also needs a majority of the three Sentinel processes to authorize a leader for the failover.

Keep the Sentinel configuration/state path persistent and writable.

8.13 Run Valkey and Sentinel

Use the pinned Valkey image or package. If containerized, use stable host networking so Sentinel advertises the real VPC addresses instead of NAT-remapped container addresses.

A per-host container intent is:

```
services:
  valkey:
    image: <PINNED_VALKEY_IMAGE>
```

```

network_mode: host
restart: unless-stopped
command: ["valkey-server", "/etc/valkey/valkey.conf"]
volumes:
  - /etc/lorawan-cloud/valkey/valkey.conf:/etc/valkey/valkey.conf:ro
  - /etc/lorawan-pki/valkey:/run/valkey-tls:ro
  - /srv/valkey/data:/data

sentinel:
  image: <PINNED_VALKEY_IMAGE>
  network_mode: host
  restart: unless-stopped
  command: ["valkey-server", "/var/lib/valkey-sentinel/sentinel.conf", "--sentinel"]
  volumes:
    - /srv/valkey/sentinel:/var/lib/valkey-sentinel
    - /etc/lorawan-pki/valkey:/run/valkey-tls:ro

```

Place the host's writable `sentinel.conf` under `/srv/valkey/sentinel/` with ownership matching the image runtime user.

Validate the pinned image's binary/config paths before start.

8.14 Verify Valkey replication and Sentinel quorum

From an approved host with `valkey-cli`, use TLS and protected password input.

Check each data node:

```

valkey-cli --tls \
--cacert <VALKEY_CA> \
-h <VALKEY_PRIVATE_IP> -p 6379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE

```

Normal initial state:

```

ha-01 -> master/primary
ha-02 -> slave/replica
ha-03 -> slave/replica

```

Then query each Sentinel:

```

valkey-cli --tls \
--cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL CKQUORUM lorawan-valkey

```

Pass only when all three Sentinels agree on the same primary and `CKQUORUM` reports enough Sentinels for quorum and failover authorization.

8.15 Validated HAProxy Valkey writable-primary routing

The live client-facing Valkey endpoints are:

```

ulc-01 -> 10.104.0.2:16379
ulc-02 -> 10.104.0.4:16379
TLS identity presented end-to-end -> valkey.internal.lorawan.com

```

Application traffic is TCP/TLS passthrough: HAProxy does **not** terminate or re-encrypt the client stream. HAProxy performs a separate TLS health connection to each Valkey backend, verifies the commissioned CA and SNI `valkey.internal.lorawan.com`, authenticates with the dedicated `haproxy-health` ACL identity loaded from a protected `systemd EnvironmentFile`, sends exact CRLF-framed `INFO replication`, and accepts only `role:master`.

The validated HAProxy 2.8 health-check shape is:

```
backend valkey_primary_backend
    mode tcp
    option tcp-check

    tcp-check connect ssl sni valkey.internal.lorawan.com
    tcp-check send-lf "AUTH haproxy-health %[env{VALKEY_HAPROXY_HEALTH_PASSWORD}]\r\n"
    tcp-check expect string +OK
    tcp-check send-lf "INFO replication\r\n"
    tcp-check expect min-recv 64 string role:master
```

The main Valkey application/replication password is **not** stored in `haproxy.cfg`. HAProxy receives only the separate least-privilege health credential. The `haproxy` service can read `/etc/haproxy/ca/valkey-ca.crt` but cannot read any Valkey node private key.

8.16 Validated Valkey endpoint and failover result

For a client on an application node, map `valkey.internal.lorawan.com` to that node's local HAProxy private IP and connect to `:16379`. A direct validation form is:

```
valkey-cli --tls \
--cacert <VALKEY_CA> \
--sni valkey.internal.lorawan.com \
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' \
INFO replication
```

Expected routed role: `master`.

The controlled Phase 8C.14 failover proved the complete path: Sentinel promoted `ulc-02` after `ulc-03` Valkey was stopped; all three Sentinels agreed; both HAProxy `:16379` endpoints followed the new primary automatically without configuration changes or reloads; ten consecutive requests through each endpoint were master-only; pre-failover data survived; post-failover writes succeeded; `ulc-03` rejoined as a replica; and the final primary reported two connected replicas. At the Phase 8C.14 acceptance checkpoint, the Valkey primary was `ulc-02 (10.104.0.4)`. Treat Sentinel as authoritative for the live elected primary and do not manually fail back merely to match this dated checkpoint.

8.17 Final acceptance and Phase 9 handoff

Phase 8 core-service acceptance is split deliberately:

MQTT infrastructure accepted: Mosquitto 2.0.18 runs on `ulc-01/02`, both brokers serve TLS on `:8884`, `10.104.0.2:8883` provides validated HAProxy TLS passthrough, the shared broker certificate name `mqtt.internal.lorawan.com` verifies, HAProxy detects a failed broker, traffic can continue through the surviving broker, and service recovery has passed. This proves the broker/TLS/failover foundation only. ChirpStack workload authentication/ACLs, gateway identities, and a two-app-node MQTT route remain explicit later dependencies.

Valkey HA accepted: one primary and two replicas are maintained; three TLS Sentinels have quorum 2; failover-safe `masterauth` exists on every node; HAProxy exposes only the writable primary through both application-node

:16379 endpoints; automatic promotion, client routing convergence, data survival, writes after promotion, and old-primary replica recovery all passed.

Phase 9 may now begin with **read-only ChirpStack preflight and dependency closure**. Do not start ChirpStack until its exact image/config schema is pinned and the remaining MQTT workload identity/ACL plus second application-node routing boundary is closed.

8.18 Phase 8 Execution Record - MQTT Foundation Deployment

Recorded deployment evidence:

```
Date:
2026-08-24

MQTT baseline discovery:

ulc-01
10.104.0.2

ulc-02
10.104.0.4

ulc-03
10.104.0.8

Before installation:
- no MQTT packages installed
- no Valkey packages installed
- no MQTT listeners
- no Valkey listeners
- no MQTT/Valkey containers
- no MQTT/Valkey services

Mosquitto deployment:

Installed:
- mosquitto
- mosquitto-clients

Verified:
- Mosquitto version: 2.0.18
- MQTT protocol support: MQTT v5.0/v3.1.1/v3.1

Service state:

ulc-01:
mosquitto.service active (running)

ulc-02:
mosquitto.service active (running)

Result:
MQTT broker foundation installed successfully on the first two application nodes.
```

Next validation steps:

1. Configure Mosquitto TLS listeners.
2. Deploy MQTT CA and broker certificates.
3. Configure MQTT authentication and ACLs.
4. Validate broker-to-client TLS.
5. Add HAProxy MQTT routing.
6. Continue with Valkey deployment.

8.19 Phase 8 MQTT TLS Foundation Precheck Record

Executed on:

```
ulc-01  
10.104.0.2
```

```
ulc-02  
10.104.0.4
```

Discovery results:

```
Mosquitto:  
- installed  
- active  
- enabled  
  
MQTT PKI:  
- /etc/lorawan-pki/mqtt does not exist yet  
  
Existing PKI:  
- PostgreSQL TLS present  
- PgBouncer TLS present  
- MQTT TLS material not issued yet
```

Current listeners before TLS configuration:

```
ulc-01:  
127.0.0.1:1883  
[::1]:1883  
  
ulc-02:  
127.0.0.1:1883  
[::1]:1883
```

Interpretation:

- Mosquitto is running in the default local-only state.
- No cloud MQTT listener is exposed.
- No TLS listener exists yet.
- No plaintext remote MQTT exposure exists.

Next execution step:

1. Issue MQTT broker certificates from the project CA.
2. Install /etc/lorawan-pki/mqtt on broker nodes.
3. Replace local-only Mosquitto configuration with TLS listener configuration.
4. Validate mutual TLS before HAProxy integration.

Operational note:

Gateway client certificate provisioning is intentionally deferred until the final onboarding stage. Cloud MQTT broker TLS identities must be completed, HAProxy routing must be validated, Valkey and ChirpStack integration must be operational, and production cloud-side testing must pass before issuing gateway certificates.

Certificate separation:

- Broker certificates remain only on cloud MQTT nodes.
- Gateway certificates are separate client identities.

- Broker private keys are never transferred to gateways.

Automation preference:

Deployment steps should use chained command blocks where safe. Validation, backup, permission checks, and installation actions should be grouped into reproducible runbooks instead of many isolated commands.

Deployment ordering decision - Gateway certificate provisioning

Gateway MQTT client certificate provisioning is intentionally moved to the final deployment stage.

Reason:

- Cloud MQTT infrastructure must be completed and validated first.
- Mosquitto TLS, HAProxy routing, ACL policies, ChirpStack integration, and monitoring must exist before issuing gateway identities.
- Gateway certificates are production client credentials and should only be generated when the receiving MQTT service is ready.
- This prevents issuing gateway credentials against an unfinished broker architecture.

Final order:

1. Cloud foundation
 - etcd
 - Spilo/Patroni PostgreSQL HA
 - HAProxy
 - PgBouncer
2. Cloud application services
 - Mosquitto TLS
 - Valkey
 - ChirpStack
 - integrations
3. Cloud validation
 - TLS verification
 - failover testing
 - ACL testing
 - backup/recovery testing
4. Final gateway onboarding
 - issue gateway client certificate
 - install gateway CA/client certificate/key
 - configure MQTT Forwarder
 - validate end-to-end uplink

Gateway certificate provisioning must not begin before the cloud MQTT stack passes acceptance testing.

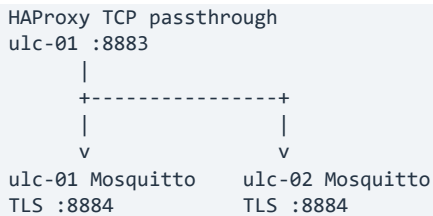
8.20 Phase 8B - MQTT HAProxy TLS Access Layer Execution Record

Objective:

Create a single MQTT TLS endpoint for gateways while keeping TLS termination on Mosquitto brokers.

Final design:

```
Gateway clients
|
| MQTT TLS :8883
|
```



Important design decision:

- HAProxy does not terminate MQTT TLS.
- Mosquitto retains broker certificates and private keys.
- HAProxy only forwards encrypted TCP traffic.
- Gateway certificates remain deferred until final onboarding.

8.20.1 HAProxy baseline check

Executed on ulc-01:

```
systemctl is-active haproxy
systemctl is-enabled haproxy
ss -H -lntp | grep haproxy
```

Existing services preserved:

```
PostgreSQL primary:
15432

PostgreSQL replicas:
15433
```

8.20.2 Initial HAProxy MQTT configuration attempt

The first configuration attempted:

```
frontend mqtt_tls
  bind 10.104.0.2:8883

backend mqtt_brokers
  server mqtt-ulc01 10.104.0.2:8883 check
  server mqtt-ulc02 10.104.0.4:8883 check
```

Validation passed:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Runtime failed because Mosquitto already owned port 8883:

```
cannot bind socket
Address already in use
```

Correction:

- HAProxy owns the external MQTT port.
- Mosquitto moves to an internal TLS port.

8.20.3 Move Mosquitto internal TLS listener

Mosquitto configuration changed:

Before:

```
listener 8883
```

After:

```
listener 8884

cafile /etc/lorawan-pki/mqtt/ca.crt
certfile /etc/lorawan-pki/mqtt/server.crt
keyfile /etc/lorawan-pki/mqtt/server.key

tls_version tlsv1.3
require_certificate false
```

Restart:

```
systemctl restart mosquitto
```

Validation:

```
ss -H -lntp | grep mosquitto
```

Expected:

```
0.0.0.0:8884
[::]:8884
```

8.20.4 Final HAProxy MQTT configuration

Backup first:

```
cp /etc/haproxy/haproxy.cfg \
/etc/haproxy/haproxy.cfg.phase8b-before-$(date -u +%Y%m%d-%H%M%S)
```

Configuration added:

```
frontend mqtt_tls
    bind 10.104.0.2:8883
    mode tcp
    option tcplog
    default_backend mqtt_brokers

backend mqtt_brokers
    mode tcp
    option tcp-check

    # The two Mosquitto brokers do not replicate live MQTT sessions/topic state.
    # Keep all normal sessions on ulc-01 so a gateway's separate uplink/downlink
    # connections and ChirpStack MQTT sessions cannot be split across brokers.
    server mqtt-ulc01 10.104.0.2:8884 check
    server mqtt-ulc02 10.104.0.4:8884 check backup
```

Validate:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Restart:

```
systemctl restart haproxy
```

Final listeners:

```
HAProxy:
10.104.0.2:8883

Mosquitto:
ulc-01 :8884
ulc-02 :8884
```

8.20.5 MQTT HA TLS validation

Client validation performed from ulc-03:

```
openssl s_client \
-connect 10.104.0.2:8883 \
-CAfile /etc/lorawan-pki/mqtt/ca.crt \
-verify_hostname mqtt.internal.lorawan.com \
-brief \
</dev/null
```

Result:

```
Protocol version: TLSv1.3
Cipher: TLS_AES_256_GCM_SHA384
Verification: OK
Verified peername: mqtt.internal.lorawan.com
```

PASS:

- HAProxy TLS passthrough
- broker certificate validation
- MQTT TLS endpoint

8.20.6 MQTT failover validation

Failure test:

Stop ulc-01 broker:

```
systemctl stop mosquitto
```

HAProxy detected failure:

```
Server mqtt_brokers/mqtt-ulc01 is DOWN
reason: Layer4 connection problem
```

Client test through HAProxy remained successful:

```
Verification: OK
```

Verified peername: mqtt.internal.lorawan.com

Recovery:

```
systemctl start mosquitto
```

HAProxy recovered the backend:

```
Server mqtt_brokers/mqtt-ulc01 is UP
reason: Layer4 check passed
2 active and 0 backup servers online
```

Phase 8B result:

MQTT HAProxy endpoint	PASS
TLS passthrough	PASS
ulc-01 Mosquitto backend	PASS
ulc-02 Mosquitto backend	PASS
Failover detection	PASS
Service recovery	PASS

8.21 Phase 8C - Valkey HA Deployment Execution Record

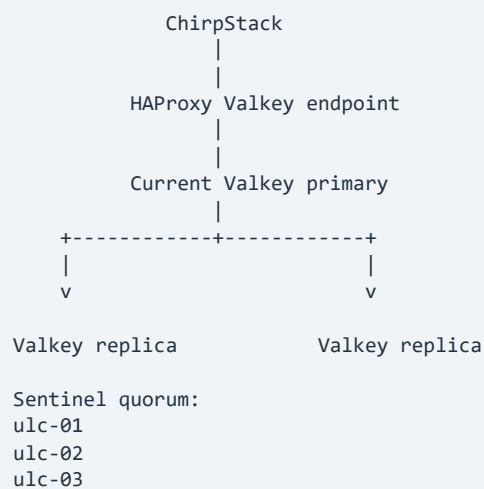
Objective:

Deploy the ChirpStack shared state backend using Valkey with future Sentinel-based high availability.

Execution boundary:

- Valkey deployment started after MQTT HA completion.
- Gateway provisioning remains deferred until cloud services are complete.
- No ChirpStack integration changes are performed until Valkey HA passes validation.
- No Git commit or push is performed during deployment sessions. Operator performs final review and commit.

Final intended architecture:



8.21.1 Valkey foundation precheck

Validated nodes:

```
ulc-01
10.104.0.2

ulc-02
10.104.0.4

ulc-03
10.104.0.8
```

Validation performed:

```
Redis/Valkey packages:
none installed

Redis/Valkey services:
none configured

Listeners:
6379 unavailable
26379 unavailable
```

Resource validation:

```
ulc-01:
RAM 1.9Gi
Available 1.3Gi

ulc-02:
RAM 1.9Gi
Available 1.4Gi

ulc-03:
RAM 1.9Gi
Available 1.4Gi
```

Result:

```
Phase 8C.1 Foundation Precheck: PASS
```

8.21.2 Valkey package installation

Deployment automation:

```
ssh \
-i /root/.ssh/cloud-deployment-phase8 \
-o IdentitiesOnly=yes \
opsadmin@NODE
```

The deployment key is required for multi-node automation. Do not rely on default SSH keys.

Installed package:

```
valkey-server
valkey-tools
```

Version installed:

Valkey 7.2.13

8.21.3 Installation status checkpoint

ulc-01

Status:

PASS

Verified:

```
valkey-server installed
service active
service enabled
```

Listener:

```
127.0.0.1:6379
[::1]:6379
```

ulc-02

Status:

PASS

Verified:

```
valkey-server installed
service active
service enabled
```

Listener:

```
127.0.0.1:6379
[::1]:6379
```

ulc-03

Status:

PASS

The first remote installation attempt failed because SSHing from ulc-03 back into opsadmin@10.104.0.8 did not provide a TTY for sudo. The corrected procedure installed Valkey locally on ulc-03 and then performed a local verification.

Verified final state:

```
Valkey version: 7.2.13
service: active
startup: enabled
```

```
listener: 127.0.0.1:6379 and [::1]:6379
PING: PONG
```

Final Phase 8C.2 foundation state:

ulc-01	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-02	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-03	Valkey 7.2.13	active	enabled	loopback:6379	PONG

Result:

```
Phase 8C.2 Valkey Foundation: PASS
```

No node exposes Valkey on the private VPC yet. Replication, authentication, TLS/private binding, Sentinel, and HAProxy routing remain intentionally deferred to Phase 8C.3 and later.

8.21.4 Next execution step

Resume from ulc-03. Do not SSH from ulc-03 back into itself for privileged package installation. The failed first attempt occurred because remote sudo on opsadmin@10.104.0.8 required a TTY. Install the remaining package locally with normal interactive sudo, then use the existing Phase 8 deployment key only to verify ulc-01 and ulc-02.

Use this single automation block:

```
sudo -v

sudo bash -euo pipefail <<'EOF'

echo '=== PHASE 8C.2 COMPLETE VALKEY FOUNDATION ==='

SSH_KEY='/root/.ssh/cloud-deployment-phase8'

echo
echo '=== STEP 1 - VERIFY THIS IS ULC-03 ==='

[ "$(hostname)" = 'ulc-03' ] || {
    echo 'FAIL: run this block on ulc-03 only'
    exit 1
}

echo 'host = ulc-03: PASS'

echo
echo '=== STEP 2 - INSTALL VALKEY LOCALLY ON ULC-03 ==='

export DEBIAN_FRONTEND=noninteractive
apt-get update
apt-get install -y valkey-server valkey-tools

systemctl enable valkey-server
systemctl start valkey-server

echo
echo '=== STEP 3 - VERIFY ULC-03 ==='

valkey-server --version
systemctl is-active valkey-server
systemctl is-enabled valkey-server
ss -H -lnt | grep -E '127\.0\.0\.1:6379|\[::1\]:6379' || {
    echo 'FAIL: ulc-03 local Valkey listener missing'
    exit 1
}

valkey-cli -h 127.0.0.1 -p 6379 PING | grep -Fxq PONG || {
    echo 'FAIL: ulc-03 Valkey PING failed'
    exit 1
}
```

```

}

echo 'ulc-03 Valkey foundation: PASS'

echo
echo '=== STEP 4 - VERIFY ALL THREE NODES ==='

for ENTRY in \
  'ulc-01|10.104.0.2' \
  'ulc-02|10.104.0.4'
do
  NAME="${ENTRY%|*}"
  IP="${ENTRY##*|}"

  echo
  echo "===== $NAME ($IP) ====="

  ssh \
    -i "$SSH_KEY" \
    -o IdentitiesOnly=yes \
    -o StrictHostKeyChecking=yes \
    "opsadmin@$IP" '
      set -e
      hostname
      valkey-server --version
      systemctl is-active valkey-server
      systemctl is-enabled valkey-server
      ss -H -lnt | grep -E "127\.0\.0\.1:6379|\[::1\]:6379"
      valkey-cli -h 127.0.0.1 -p 6379 PING
    '
done

echo
echo '===== ulc-03 (10.104.0.8) ====='
hostname
valkey-server --version
systemctl is-active valkey-server
systemctl is-enabled valkey-server
ss -H -lnt | grep -E "127\.0\.0\.1:6379|\[::1\]:6379"
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.2 VALKEY FOUNDATION: PASS ==='
echo 'All three nodes have Valkey installed, active, enabled, local-only, and responding to PING.'
echo 'STOP HERE. Do not expose 6379 or configure replication until Phase 8C.3 security configuration.'

EOF

```

The block performs four clear actions:

1. Confirms it is running on ulc-03.
2. Installs and starts Valkey locally on ulc-03, avoiding the self-SSH sudo problem.
3. Verifies version, service state, loopback-only listener, and PONG on ulc-03.
4. Verifies the same accepted foundation state on ulc-01 and ulc-02 through the retained deployment SSH key.

Expected final foundation state:

ulc-01	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-02	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-03	Valkey 7.2.13	active	enabled	loopback:6379	PONG

Latest verified execution evidence:

```

ulc-01 (10.104.0.2)
Server v=7.2.13
active
enabled

```

```

127.0.0.1:6379 LISTEN
[::1]:6379 LISTEN
PING -> PONG
PASS

ulc-02 (10.104.0.4)
Server v=7.2.13
active
enabled
127.0.0.1:6379 LISTEN
[::1]:6379 LISTEN
PING -> PONG
PASS

```

At this checkpoint, ulc-01 and ulc-02 are definitively accepted. Do not mark Phase 8C.2 complete until the Step 3/Step 5 output for ulc-03 also shows Valkey 7.2.13, active, enabled, loopback-only 6379, and PONG.

After all three pass, continue to Phase 8C.3 in this order:

1. Run the security/TLS preflight below on all three nodes.
2. Issue and deploy Valkey server certificates only after confirming the actual Ubuntu package paths, service account, and TLS capability.
3. Configure authentication plus private-network/TLS listeners.
4. Configure one primary and two replicas.
5. Deploy three Sentinel members with quorum 2.
6. Add HAProxy writable-primary routing.
7. Perform controlled Valkey failover and recovery testing.

8.21.5 Phase 8C.3 security/TLS preflight

Observed preflight checkpoint:

```

ulc-01: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf, loopback 6379, OpenSSL linked, PONG
ulc-02: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf, loopback 6379, OpenSSL linked, PONG
ulc-03: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf; output was interrupted by the systemctl pager
at "lines 1-4/4 (END)"

```

The ulc-03 interruption is not a Valkey failure. `systemctl show` opened the pager because this portion was executed locally in an interactive terminal. Press `q` to leave the pager, then run the no-pager continuation below. Do not mutate authentication, TLS listeners, replication, or Sentinel until the ulc-03 remainder is captured.

```

sudo bash -euo pipefail <<'EOF'

echo '=== PHASE 8C.3 ULC-03 PREFLIGHT CONTINUATION ==='

[ "$(hostname)" = 'ulc-03' ] || {
    echo 'FAIL: run on ulc-03 only'
    exit 1
}

echo
echo '--- systemd service identity ---'
SYSTEMD_PAGER=cat systemctl show valkey-server --no-pager \
    -p User \
    -p Group \
    -p FragmentPath \
    -p ExecStart

echo
echo '--- package configuration files ---'
dpkg -L valkey-server |
    grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true

```

```

echo
echo '--- configured security/persistence directives ---'
grep -En \
'^[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-file|tls-auth-
clients|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' \
/etc/valkey/valkey.conf || true

echo
echo '--- relevant defaults including commented directives ---'
grep -En \
'^[[:space:]]*#[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|tls-auth-clients|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' \
/etc/valkey/valkey.conf | head -80 || true

echo
echo '--- current listener ---'
ss -H -lntp | grep ':6379' || true

echo
echo '--- TLS capability evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null |
grep -Ei 'ssl|crypto' || true

echo
echo '--- local PING ---'
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.3 ULC-03 PREFLIGHT CONTINUATION COMPLETE ==='

EOF

```

Acceptance for this continuation is: service identity valkey:valkey, /etc/valkey/valkey.conf confirmed, loopback-only 6379, OpenSSL libraries linked, and PONG.

This step is read-only. It discovers the actual Valkey service account, configuration files, current bind/persistence settings, and TLS support before any security or replication mutation.

```

sudo -v
sudo bash -euo pipefail <<'EOF'
SSH_KEY='/root/.ssh/cloud-deployment-phase8'

echo '=== PHASE 8C.3 VALKEY SECURITY/TLS PREFLIGHT ==='

for ENTRY in 'ulc-01|10.104.0.2' 'ulc-02|10.104.0.4'; do
    NAME="${ENTRY%|*}"
    IP="${ENTRY##*|}"
    ssh -i "$SSH_KEY" -o IdentitiesOnly=yes -o StrictHostKeyChecking=yes "opsadmin@$IP" "bash -s -- '$NAME'"
<<'REMOTE'
set -euo pipefail
NAME="$1"
echo
echo "===== $NAME ====="
hostname
echo '--- package/version ---'
dpkg-query -W -f='${Status}|${Version}\n' valkey-server valkey-tools
valkey-server --version
echo '--- systemd ---'
systemctl show valkey-server -p User -p Group -p FragmentPath -p ExecStart
echo '--- config files ---'
dpkg -L valkey-server | grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true
echo '--- active directives ---'
grep -Eh '^[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' /etc/valkey/*.conf /etc/valkey/*.conf.d/*
2>/dev/null || true
echo '--- listener ---'
ss -H -lntp | grep ':6379' || true
echo '--- TLS evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null | grep -Ei 'ssl|crypto' || true
valkey-server --help 2>&1 | grep -i tls || true
echo '--- ping ---'

```

```

valkey-cli -h 127.0.0.1 -p 6379 PING
REMOTE
done

echo
echo '==== ulc-03 ==== '
hostname
echo '--- package/version ---'
dpkg-query -W -f='${Status}]${Version}\n' valkey-server valkey-tools
valkey-server --version
echo '--- systemd ---'
systemctl show valkey-server -p User -p Group -p FragmentPath -p ExecStart
echo '--- config files ---'
dpkg -L valkey-server | grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true
echo '--- active directives ---'
grep -Eh '^[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' /etc/valkey/*.conf /etc/valkey/*.conf.d/*
2>/dev/null || true
echo '--- listener ---'
ss -H -lntp | grep ':6379' || true
echo '--- TLS evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null | grep -Ei 'ssl|crypto' || true
valkey-server --help 2>&1 | grep -i tls || true
echo '--- ping ---'
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.3 PREFLIGHT COMPLETE ==='
EOF

```

Do not expose port 6379, set passwords, configure replication, or deploy Sentinel until this preflight output has been reviewed.

8.21.5 Phase 8C.3 security/TLS preflight result

The preflight passed on all three nodes.

Validated baseline:

```

Valkey server/tools: 7.2.13+dfsg1-0ubuntu0.1
Service user/group: valkey:valkey
Main config: /etc/valkey/valkey.conf
Systemd unit: /usr/lib/systemd/system/valkey-server.service
Current listener: loopback-only 6379
Local connectivity: PONG
TLS libraries: libssl.so.3 + libcrypto.so.3

```

The explicitly visible active baseline on ulc-03 is:

```

bind 127.0.0.1 -::1
protected-mode yes
port 6379
dbfilename dump.rdb
dir /var/lib/valkey
appendonly no

```

Result:

```

Phase 8C.3 security/TLS preflight: PASS

```

8.21.6 Phase 8C.4 Valkey TLS certificate issuance result

Certificate issuance completed successfully on ulc-03 from the existing internal CA.

Issuance directory:

```
/root/lorawan-pg-ca/valkey-issuance-20260825-003932
```

CA identity:

```
subject=CN = LoRaWAN PostgreSQL Internal CA
issuer=CN = LoRaWAN PostgreSQL Internal CA
SHA-256
fingerprint=99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E
```

Issued node identities:

```
ulc-01 -> valkey.internal.lorawan.com + ulc-01 + 10.104.0.2
ulc-02 -> valkey.internal.lorawan.com + ulc-02 + 10.104.0.4
ulc-03 -> valkey.internal.lorawan.com + ulc-03 + 10.104.0.8
```

Each certificate includes both `serverAuth` and `clientAuth` extended key usage so the same per-node identity can participate in TLS-protected Valkey replication and later Sentinel communication without sharing one private key across hosts.

Certificate-chain verification passed for all three nodes. Certificate/private-key public-key hashes also matched for all three nodes.

Recorded SHA-256 certificate fingerprints:

```
ulc-01 27:17:81:BA:E3:9A:8D:F8:AF:C0:CD:DA:1A:2D:DB:25:27:84:C4:65:B3:4D:B9:40:7A:32:89:F7:EB:AF:AC:0A
ulc-02 BD:DD:EF:C9:95:C7:CF:36:50:77:CE:42:38:97:B0:DB:77:CB:F3:2C:50:AD:41:98:9C:AA:D6:84:0A:43:17:A6
ulc-03 E5:5C:A8:BE:E1:C7:0D:9E:A8:5C:93:3B:4B:80:EF:B8:58:FD:47:FA:08:8C:5E:3C:6D:94:C4:4C:9F:B0:4B:DC
```

Result:

```
Phase 8C.4 Valkey TLS certificate issuance: PASS
```

Keep the complete issuance directory until the Valkey/Sentinel HA layer and final security cleanup are complete. Do not delete the deployment SSH key or node private keys during commissioning.

8.21.7 Phase 8C.5 Valkey TLS material installation checkpoint

The remote TLS material installation completed successfully on `ulc-01` and `ulc-02`.

Validated on both nodes:

```
/etc/lorawan-pki/valkey          750 root:valkey
/etc/lorawan-pki/valkey/ca.crt    640 root:valkey
/etc/lorawan-pki/valkey/server.crt 640 root:valkey
/etc/lorawan-pki/valkey/server.key 640 root:valkey

ca-read-PASS
cert-read-PASS
key-read-PASS
server.crt: OK
```

Observed file sizes on `ulc-01` and `ulc-02`:

```
ca.crt      1891 bytes
server.crt  1627 bytes
```

```
server.key 1704 bytes
```

Final Phase 8C.5 status:

```
ulc-01 TLS material: PASS
ulc-02 TLS material: PASS
ulc-03 TLS material: PASS
```

ulc-03 local verification confirmed:

```
/etc/lorawan-pki/valkey          750 root:valkey
/etc/lorawan-pki/valkey/ca.crt    640 root:valkey (1891 bytes)
/etc/lorawan-pki/valkey/server.crt 640 root:valkey (1627 bytes)
/etc/lorawan-pki/valkey/server.key 640 root:valkey (1708 bytes)
ca-read-PASS
cert-read-PASS
key-read-PASS
/etc/lorawan-pki/valkey/server.crt: OK
certificate/private-key hash match: PASS
```

The Valkey service was deliberately left unchanged after PKI staging. ulc-03 remained active, listening only on loopback plaintext 6379, and returned PONG. This proves certificate installation itself did not alter runtime behavior.

Result:

```
Phase 8C.5 Valkey TLS material installation: PASS
```

8.21.8 Phase 8C.6 Valkey TLS + authentication result

Phase 8C.6 passed on all three nodes.

Validated runtime state:

```
ulc-01 10.104.0.2:6379 TLS-only + password auth: PASS
ulc-02 10.104.0.4:6379 TLS-only + password auth: PASS
ulc-03 10.104.0.8:6379 TLS-only + password auth: PASS
```

Security checks passed on every node:

```
service active + enabled
TLSv1.3 negotiated
cipher TLS_AES_256_GCM_SHA384
CA verification OK
peer name valkey.internal.lorawan.com verified
plaintext connection rejected
TLS without password rejected with NOAUTH
TLS with password returned PONG
tls-replication yes prepared
```

The shared Valkey authentication secret is retained only in the root-owned secret file created during commissioning and was not printed in the execution record.

Result:

```
Phase 8C.6 Valkey TLS + authentication: PASS
```

8.21.9 Phase 8C.7 initial replication attempt and authentication correction

The first Phase 8C.7 replication block stopped before any replica configuration was changed. Its primary precheck attempted to authenticate `valkey-cli` using the `VALKEYCLI_AUTH` environment variable, but the observed Ubuntu package behavior returned:

```
exit_code=0
NOAUTH Authentication required.
```

This is a command-client authentication issue, not a Valkey TLS or server failure. Phase 8C.6 already proved authenticated TLS PONG using the explicit `-a` option. The replication attempt stopped at Step 1, so `ulc-01/02/03` remain in the Phase 8C.6 standalone TLS-only authenticated state and no `replicaof` directive has yet been applied.

Correction: all remaining commissioning commands use explicit `-a "$AUTH"` for `valkey-cli` rather than relying on the environment-variable path. The secret remains unprinted and root-only. Valkey's upstream CLI documentation supports both `-a` and `VALKEYCLI_AUTH`, but actual observed package behavior is authoritative for this deployment.

The corrected Phase 8C.7 replication deployment then passed.

Final topology:

```
ulc-01 10.104.0.2 = PRIMARY
ulc-02 10.104.0.4 = REPLICa
ulc-03 10.104.0.8 = REPLICa
```

Observed primary state:

```
role:master
connected_slaves:2
slave0:ip=10.104.0.4,port=6379,state=online,offset=14,lag=0
slave1:ip=10.104.0.8,port=6379,state=online,offset=14,lag=0
```

Both replicas reported `role:slave`, `master_host:10.104.0.2`, `master_port:6379`, and `master_link_status:up`. Primary write propagation passed to both replicas, and both replicas correctly rejected direct writes with `READONLY`.

Result:

```
Phase 8C.7 Valkey TLS replication: PASS
```

8.21.10 Phase 8C.8 Sentinel foundation precheck checkpoint

The Sentinel foundation precheck has started. `ulc-01` completed successfully and showed:

```
no separate Sentinel package installed
valkey-sentinel package candidate: 7.2.13+dfsg1-0ubuntu0.1
redis-sentinel package candidate also exists but is not selected
/usr/bin/valkey-server present
no Sentinel systemd unit currently installed
valkey-server supports Sentinel mode via --sentinel
port 26379 free
```

This confirms the correct family is the matching `valkey-sentinel 7.2.13` package, not the older Redis Sentinel package.

The first continuation attempt reached `ulc-02` and confirmed:

```
valkey-sentinel candidate: 7.2.13+dfsg1-0ubuntu0.1
valkey-sentinel not installed
/usr/bin/valkey-server present
valkey-server supports Sentinel mode via --sentinel
```

A second continuation completed the remaining ulc-02 checks:

```
no Sentinel systemd unit
26379 free
```

ulc-02 therefore matches the validated ulc-01 Sentinel foundation baseline: matching valkey-sentinel 7.2.13 candidate, package not installed, Sentinel mode supported by valkey-server, no Sentinel unit yet, and port 26379 free.

The ulc-03 precheck then confirmed the host, matching valkey-sentinel candidate 7.2.13+dfsg1-0ubuntu0.1, package-not-installed state, /usr/bin/valkey-server, and visible Sentinel-mode support. The script stopped immediately after printing the Sentinel-mode help text, before the explicit PASS line, systemd-unit check, and port 26379 check.

The next ulc-03 continuation also stopped at the Sentinel capability step before any package/config/service mutation. The command used `HELP="$(valkey-server --help 2>&1)"` while the wrapper was running with `set -e`. On this Ubuntu Valkey build, valkey-server --help prints valid help text but exits non-zero, so the command substitution assignment itself triggers `set -e` and terminates the script before the captured text can be checked. The visible earlier output already proves that the help text contains `Sentinel mode;`; this is a shell-control-flow issue, not a Sentinel capability failure.

The corrected ulc-03 continuation passed. Final Phase 8C.8 Sentinel foundation precheck state:

```
ulc-01: PASS
ulc-02: PASS
ulc-03: PASS
valkey-sentinel candidate = 7.2.13+dfsg1-0ubuntu0.1
Sentinel package not installed
valkey-server supports --sentinel
no Sentinel systemd unit
26379 free
```

Result:

```
Phase 8C.8 Sentinel foundation precheck: PASS
```

Phase 8C.9 installed and inspected the matching valkey-sentinel package on all three nodes. Observed package version on ulc-01, ulc-02, and ulc-03: 7.2.13+dfsg1-0ubuntu0.1.

Package/runtime evidence:

```
executable: /usr/bin/valkey-sentinel
service: /usr/lib/systemd/system/valkey-sentinel.service
ExecStart: /usr/bin/valkey-sentinel /etc/valkey/sentinel.conf --supervised systemd --daemonize no
User: valkey
Group: valkey
configuration: /etc/valkey/sentinel.conf
configuration permissions: 640 valkey:valkey
RuntimeDirectory: sentinel
ReadWriteDirectories: /etc/valkey
```

The packaged default configuration is not suitable for production as-is:

```
protected-mode no
port 26379
sentinel monitor mymaster 127.0.0.1 6379 2
```

Immediately after package installation, valkey-sentinel was stopped and disabled on every node. Final safety verification passed on all three nodes:

```
ActiveState=inactive
UnitFileState=disabled
26379 closed: PASS
```

Result:

```
Phase 8C.9 Sentinel package foundation: PASS
```

No Sentinel quorum is active yet. Do not start or enable the service with the packaged default configuration.

Phase 8C.10 Sentinel TLS configuration preflight passed on ulc-03. Sentinel remained inactive and disabled, and port 26379 remained closed. The packaged Sentinel configuration exposes only the default plaintext port 26379 directive and the default sentinel monitor mymaster 127.0.0.1 6379 2; no TLS directives are pre-populated in /etc/valkey/sentinel.conf.

The valkey service account can read all staged TLS files (ca.crt, server.crt, and server.key, each 640 root:valkey). The current Valkey topology was also reconfirmed before Sentinel configuration:

```
role:master
connected_slaves:2
```

The existing writable primary remains 10.104.0.2:6379 with both replicas connected.

Valkey's TLS documentation confirms that Sentinel inherits the common TLS directives and that tls-replication yes controls both Sentinel-to-Valkey TLS and TLS on Sentinel's own inter-Sentinel listener. Therefore the production Sentinel configuration will explicitly disable plaintext with port 0, enable tls-port 26379, set the staged certificate/key/CA paths, set tls-replication yes, and retain quorum 2.

Result:

```
Phase 8C.10 Sentinel TLS configuration preflight: PASS
```

Phase 8C.11 TLS-only Sentinel quorum deployment progressed successfully through quorum validation. All three Sentinels started with node-specific private binds, plaintext disabled (port 0), TLS-only tls-port 26379, the staged certificate/key/CA files, tls-auth-clients no, tls-replication yes, separate Sentinel password authentication, and monitoring of lorawan-valkey at the current primary 10.104.0.2:6379 with quorum 2.

Validated runtime evidence:

```
ulc-01 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.2:26379
ulc-02 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.4:26379
ulc-03 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.8:26379
TLSv1.3 handshake to all three: PASS
CA verification: OK
peer name valkey.internal.lorawan.com: verified
all three Sentinels report primary 10.104.0.2:6379
two Sentinel peers discovered: PASS
CKQUORUM: OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

The enablement continuation then confirmed ulc-03 was successfully enabled and remained active. Final service-state verification also passed on ulc-01 and ulc-02:

```
ulc-03 service=active, enabled=enabled
ulc-01 service=active, enabled=enabled, listeners 127.0.0.1:26379 + 10.104.0.2:26379
ulc-02 service=active, enabled=enabled, listeners 127.0.0.1:26379 + 10.104.0.4:26379
```

The final Sentinel verification continuation passed completely. ulc-03 exposes the expected TLS-only listeners on 127.0.0.1:26379 and 10.104.0.8:26379; all three Sentinel members report the same current primary 10.104.0.2:6379; ulc-03 sees exactly two Sentinel peers; and SENTINEL CKQUORUM lorawan-valkey returns OK 3 usable Sentinels. Quorum and failover authorization can be reached.

Final Phase 8C.11 state:

```
ulc-01 Sentinel = active + enabled
ulc-02 Sentinel = active + enabled
ulc-03 Sentinel = active + enabled
Sentinel transport = TLS-only :26379
Sentinel members = 3
quorum = 2
current primary = 10.104.0.2:6379
peer_count = 2
primary discovery = PASS on all three members
CKQUORUM = PASS
```

Result:

```
Phase 8C.11 TLS Sentinel quorum: PASS
```

Phase 8C.12 controlled automatic failover testing started. The starting-state check passed with Sentinel reporting 10.104.0.2:6379 as the primary, CKQUORUM returned OK 3 usable Sentinels, and the pre-failover test write to ulc-01 succeeded.

The script then stopped immediately at Step 4 while attempting to run `sudo systemctl stop valkey-server` remotely on ulc-01 over SSH. No output confirmed that the service actually stopped, and no automatic promotion output followed. Therefore automatic failover has **not yet been proven** and the test must continue from a verified current-state check rather than assuming ulc-01 went down.

This matches the earlier commissioning pattern where remote `sudo` from ulc-03 can terminate a non-interactive SSH block when `sudo` requires a terminal/password path. The correction is to first inspect ulc-01's actual Valkey service state and Sentinel's current master view. If ulc-01 is still active, stop it with an SSH allocation that permits the remote `sudo` interaction, while leaving `valkey-sentinel` running. If it is already inactive, do not stop it again. Then continue observing Sentinel without manually changing replication.

The corrected Phase 8C.12 continuation immediately observed that Sentinel's current primary had already changed to 10.104.0.8 (ulc-03):

```
=== PHASE 8C.12 AUTOMATIC FAILOVER CONTINUATION ===
=== STEP 1 - DISCOVER CURRENT STATE ===
Sentinel current primary = 10.104.0.8
```

This is strong evidence that the previous failure injection actually succeeded before the shell returned: ulc-01 stopped long enough for Sentinel to complete an automatic election and promote ulc-03. Do not stop any Valkey node again and do not manually issue `replicaof`. The remaining work is recovery verification, not another failover trigger.

Phase 8C.12 recovery verification then advanced substantially. All three Sentinels agreed on ulc-03 (10.104.0.8) as the elected primary. ulc-03 reported `role:master` with ulc-02 already online as a replica. The original pre-failover test key `phase8c12:failover:1787621601` was still present on ulc-03 with its expected `before-*` value, proving the pre-failover write survived the automatic promotion. A fresh authenticated TLS write to ulc-03 also succeeded.

Observed checkpoint:

```
ulc-01 Sentinel reports primary=10.104.0.8
ulc-02 Sentinel reports primary=10.104.0.8
ulc-03 Sentinel reports primary=10.104.0.8
three-Sentinel consensus: PASS
ulc-03 role:master
connected_slaves:1
ulc-02 online as replica
pre-failover data survived: PASS
new-primary write: PASS
```

The pasted output then ended at STEP 6 - INSPECT OLD PRIMARY ULC-01 before the ulc-01 service-state result was printed. A later minimized recovery block also stopped immediately at STEP 1 - CHECK ULC-01 SSH + VALKEY STATE with no remote output. This means the remaining problem is specifically the ulc-03 -> ulc-01 SSH inspection path, not the already-proven Sentinel failover. Do not trigger another failover and do not modify replication manually.

Phase 8C.12 is now proven through automatic promotion, Sentinel consensus, data survival, and successful writes on the promoted primary.

The follow-up SSH diagnostic to ulc-01 also completed successfully. SSH authentication with `/root/.ssh/cloud-deployment-phase8` is healthy and the remote host key matches. The remote service state is:

```
host=ulc-01
valkey=inactive
sentinel=active
ssh exit status=0
```

This confirms the original failure injection succeeded exactly as intended: only `valkey-server` on ulc-01 is down, while the local Sentinel stayed alive and participated in the election. There is no SSH or host reachability problem.

The recovery step then successfully started `valkey-server` on ulc-01. Immediate service verification showed:

```
valkey=active
sentinel=active
```

No replication directive was changed manually and ulc-03 remains the Sentinel-elected primary.

The final read-only recovery verification is currently waiting for ulc-01 to be reconfigured by Sentinel after its service restart. The observed output has reached at least attempt 18 of the bounded 60-attempt loop without yet matching all required replica fields. This is not yet a failure because the script intentionally allows up to 120 seconds (60 attempts at 2-second intervals) for recovery convergence.

The bounded rejoin wait did not immediately converge, so a read-only Sentinel diagnostic was captured. Sentinel still reports ulc-03 (10.104.0.8:6379) as the healthy master, with `num-slaves=2`, `num-other-sentinels=2`, quorum 2, and failover epoch 1.

Sentinel's replica view now proves that ulc-01 has been logically converted to a replica of ulc-03:

```
ulc-02 10.104.0.4:6379
flags=slave
master-host=10.104.0.8
master-port=6379
master-link-status=ok
```

```
ulc-01 10.104.0.2:6379
flags=slave
master-host=10.104.0.8
master-port=6379
master-link-status=err
```

The ulc-01 Sentinel log confirms the automatic recovery action itself occurred:

```
-sdown slave 10.104.0.2:6379 ... @ lorawan-valkey 10.104.0.8 6379
+convert-to-slave slave 10.104.0.2:6379 ... @ lorawan-valkey 10.104.0.8 6379
```

Therefore Sentinel failover and automatic topology rewriting are proven. The only remaining recovery defect is the data-plane replication link from ulc-01 to the elected primary: Sentinel sees ulc-01 as a slave of 10.104.0.8, but reports master-link-status=err.

The first read-only replication-link diagnostic captured the following ulc-01 state:

```
role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:down
master_last_io_seconds_ago:-1
master_sync_in_progress:1
master_sync_total_bytes:-1
master_sync_read_bytes:0
master_sync_left_bytes:-1
connected_slaves:0
```

This proves ulc-01 has accepted the Sentinel-directed replica role and is attempting an initial/full synchronization, but the data connection has not progressed far enough to receive any bytes. The diagnostic then stopped at the configuration-inspection step because opsadmin cannot read /etc/valkey/valkey.conf directly:

```
grep: /etc/valkey/valkey.conf: Permission denied
```

That permission error is from the diagnostic command only; it is not evidence of a Valkey runtime failure. The configuration file is intentionally restricted.

The corrected privileged config read then exposed an important clue on ulc-01:

```
replica-read-only yes
bind 127.0.0.1 10.104.0.2
port 0
tls-port 6379
tls-cert-file /etc/lorawan-pki/valkey/server.crt
tls-key-file /etc/lorawan-pki/valkey/server.key
tls-ca-cert-file /etc/lorawan-pki/valkey/ca.crt
tls-auth-clients no
tls-replication yes
replicaof 10.104.0.8 6379
```

masterauth is notably absent from the active replication/security directives even though the Valkey primary requires password authentication. That is a strong candidate explanation for the stalled full sync: Sentinel correctly rewrote replicaof to the newly elected primary, but the restarted former primary may no longer possess the credential needed for replica-to-primary authentication.

The direct TLS test in the same block failed only because it was run as opsadmin, which cannot read /etc/lorawan-pki/valkey/ca.crt (root:valkey 640). TCP reachability to 10.104.0.8:6379 passed, so that CA permission error was a diagnostic-user artifact, not proof of a TLS transport failure.

The follow-up confirmation then proved the transport and primary credentials are healthy:

```
masterauth/masteruser absent from /etc/valkey/valkey.conf
TLSv1.3 handshake from ulc-01 to 10.104.0.8:6379: PASS
CA verification: OK
Verified peername: valkey.internal.lorawan.com
authenticated valkey-cli PING from ulc-01 to ulc-03: PONG
ulc-01 role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:down
```

The attempted runtime `CONFIG GET masterauth` returned `NOAUTH Authentication required` because that local diagnostic command itself did not authenticate, so it does not establish the runtime value. However, the persistent configuration definitely lacks `masterauth`, while a manually authenticated TLS client from the same host reaches the elected primary successfully. Combined with the stalled replica link, this isolates the commissioning defect to the missing replica authentication credential rather than network, CA, hostname verification, or primary availability.

Root cause: Phase 8C.7 placed `masterauth` only on the nodes that were replicas at that moment. `ulc-01` was the original primary and therefore did not receive a persistent `masterauth`. After automatic failover, Sentinel correctly changed `ulc-01` to follow `ulc-03`, but Sentinel's topology rewrite does not supply the persistent Valkey replication password. For failover-safe symmetry, every Valkey node must retain the same `masterauth` even while it is primary; the directive is inert while a node is primary and becomes necessary if that node later becomes a replica.

The `masterauth` repair was then applied successfully on `ulc-01`. A timestamped backup was created at `/etc/valkey/valkey.conf.phase8c12-masterauth-before-20260825-020750`; the Sentinel-selected `replicaof 10.104.0.8 6379` directive was explicitly verified and preserved; exactly one persistent `masterauth` directive was written; `CONFIG SET masterauth` returned `OK`; and an authenticated `CONFIG GET masterauth` matched the expected secret without printing it.

Observed repair evidence:

```
replicaof 10.104.0.8 6379
masterauth lines=1
persistent masterauth: PASS
runtime masterauth: PASS
runtime masterauth verification: PASS
ulc-01 masterauth repair complete
```

The combined automation then dropped into an interactive root shell on `ulc-01` instead of returning cleanly to the outer `ulc-03` script, so the remaining verification was rerun as a separate continuation from `ulc-03`. This was an orchestration/TTY here-document issue, not a repair failure.

The post-repair verification immediately confirmed that the missing `masterauth` was the actual recovery defect. On the first check after applying the credential, `ulc-01` reported:

```
attempt=1 link=up sync=0
role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:up
master_last_io_seconds_ago:1
master_sync_in_progress:0
ulc-01 replication recovery: PASS
ulc-01 persistent masterauth: PASS
```

This is direct causal evidence: no `replicaof` change, service restart, or fallback was performed; adding the correct persistent/runtime `masterauth` alone restored the replica link to the Sentinel-elected primary.

The next all-node symmetry audit then failed on `ulc-02`:

```
FAIL: ulc-02 masterauth mismatch
```

Therefore Phase 8C.12 recovery is not yet fully closed. ulc-01 is healthy again, but failover-safe authentication is not normalized across all three Valkey nodes. The correct design is for every node to retain the same masterauth even when currently primary, because any node can later be converted into a replica by Sentinel.

The final failover-safe masterauth normalization then completed successfully on ulc-02 and ulc-03 without restarting Valkey and without altering Sentinel-selected topology. ulc-02 preserved replicaof 10.104.0.8 6379, received exactly one persistent masterauth, accepted the same value at runtime, and passed secret-equality verification without printing the credential. ulc-03 remained role:master, received the same persistent/runtime masterauth, and therefore all three nodes now carry the credential required if Sentinel later converts any current or former primary into a replica.

Final recovered topology:

```
ulc-03 10.104.0.8 = PRIMARY
ulc-01 10.104.0.2 = REPLICAS -> 10.104.0.8:6379, master_link_status:up
ulc-02 10.104.0.4 = REPLICAS -> 10.104.0.8:6379, master_link_status:up
ulc-03 connected_slaves:2
```

Final replication evidence:

```
slave0:ip=10.104.0.4,port=6379,state=online,offset=623818,lag=0
slave1:ip=10.104.0.2,port=6379,state=online,offset=623536,lag=0
primary write: PASS
ulc-01 write propagation: PASS
ulc-02 write propagation: PASS
```

All three Sentinels continued to report 10.104.0.8 as the primary, and SENTINEL CKQUORUM lorawan-valkey returned:

```
OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

Phase 8C.12 is therefore fully closed:

```
Automatic Sentinel promotion = PASS
Pre-failover data survival = PASS
New-primary writes = PASS
ulc-01 masterauth repair = PASS
ulc-01 replication recovery = PASS
ulc-02 masterauth normalization = PASS
ulc-03 masterauth normalization = PASS
Failover-safe masterauth on every node = PASS
One primary + two replicas = PASS
Final write propagation = PASS
Three-Sentinel consensus = PASS
Quorum after recovery = PASS
```

Result:

```
Phase 8C.12 automatic failover + recovery: PASS
```

Keep ulc-03 as the Sentinel-elected primary. Do not manually fail back to ulc-01.

Next: Phase 8C.13 HAProxy writable-primary routing. Before mutation, preflight the existing HAProxy 2.8 deployment on the application nodes, confirm port 16379 is free, confirm the HAProxy service identity and CA-read path, and validate the exact TLS health-check syntax against the installed HAProxy build. The Valkey node

certificates contain the shared SAN `valkey.internal.lorawan.com`; do not use an unissued `valkey-ha.internal.lorawan.com` name for backend certificate verification.

The first Phase 8C.13 read-only preflight reconfirmed the Valkey/Sentinel baseline before inspecting HAProxy:

```
ulc-03 role:master
connected_slaves:2
ulc-01 + ulc-02 replicas online
Sentinel primary: 10.104.0.8:6379
Valkey topology: PASS
Sentinel primary: PASS
```

The script then reached `ulc-01` successfully but stopped at the HAProxy package-version command with:

```
bash: line 12: Status: unbound variable
```

Cause: the remote shell was running `set -u` and the `dpkg-query -f='${Status}|${Version}\n'` format string used double quotes. Bash attempted to expand `${Status}` and `${Version}` as shell variables before `dpkg-query` could interpret them, and the unset `Status` variable terminated the preflight. No HAProxy configuration, listener, or service state was changed.

Correction: quote the `dpkg-query` format with single quotes (`-f='${Status}|${Version}\n'`) inside the remote shell. Resume the Phase 8C.13 preflight from the HAProxy-node inspection; there is no need to repeat the already-passed Valkey/Sentinel baseline unless topology changes before the retry.

The corrected continuation then advanced through `ulc-01` and exposed the next concrete preflight issue. HAProxy is the expected pinned build `2.8.16-0ubuntu0.24.04.3`, active and enabled, with existing PostgreSQL listeners `10.104.0.2:15432/15433`, MQTT frontend `10.104.0.2:8883`, and Mosquitto backend listener `8884`; target port `16379` is free. `/etc/haproxy/haproxy.cfg` is `0644 root:root` and the expected existing PostgreSQL/MQTT sections remain present.

The Valkey CA cannot be traversed by the HAProxy service account because `/etc/lorawan-pki/valkey` is `0750 root:valkey`; `namei` therefore ends with `ca.crt - Permission denied for non-valkey users`. Do **not** add `haproxy` to the `valkey` group, because that group can also read the Valkey server private key (`0640 root:valkey`). Instead, install a CA-only copy for HAProxy under a dedicated HAProxy-readable path such as `/etc/haproxy/ca/valkey-ca.crt`, with no server certificate or private key copied. Validate certificate equality by SHA-256 before use.

The CA-only HAProxy trust-path correction then completed successfully on both application nodes. The commissioned Valkey CA hash on `ulc-03` is `6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`, and the copied CA on both `ulc-01` and `ulc-02` matched this SHA-256 exactly.

Installed trust boundary on both HAProxy nodes:

```
/etc/haproxy/ca          750 root:haproxy
/etc/haproxy/ca/valkey-ca.crt 640 root:haproxy
HAProxy CA read: PASS
Valkey private key remains inaccessible to HAProxy: PASS
CA equality: PASS
```

The original `/etc/lorawan-pki/valkey` permissions were not weakened and the HAProxy account was not added to the `valkey` group. This preserves separation between CA trust and private-key access.

The remaining HAProxy preflight evidence passed on both `ulc-01` and `ulc-02`:

```
haproxy 2.8.16-0ubuntu0.24.04.3
service active + enabled
16379 free
```

```
existing PostgreSQL/MQTT listeners preserved
current /etc/haproxy/haproxy.cfg syntax valid
HAProxy dedicated CA readable
10.104.0.2:6379 reachable
10.104.0.4:6379 reachable
10.104.0.8:6379 reachable
```

The pasted execution ended after the ulc-02 TCP-backend reachability checks, before the separate all-backend TLS hostname-verification section printed. Therefore the CA trust staging and TCP preflight are accepted, but Phase 8C.13 is not yet closed.

The TLS identity and current-role portion of the next Phase 8C.13 preflight passed completely. All three Valkey backends negotiated TLS 1.3 with TLS_AES_256_GCM_SHA384, verified against the commissioned CA, and matched the shared certificate name `valkey.internal.lorawan.com`. Runtime role checks reconfirmed ulc-01 and ulc-02 as replicas following 10.104.0.8:6379 with links up, and ulc-03 as `role:master` with two connected replicas.

The first offline HAProxy TLS-backend syntax test failed before any production mutation. The temporary test config split one server directive across multiple physical lines without continuation syntax:

```
server test-primary
  10.104.0.8:6379
  check
  ssl
  verify required
  ca-file /etc/haproxy/ca/valkey-ca.crt
  sni str(valkey.internal.lorawan.com)
```

HAProxy 2.8 therefore parsed each subsequent line as a separate backend keyword and reported unknown keyword `'10.104.0.8:6379'`, followed by equivalent errors for `check`, `ssl`, `verify`, `ca-file`, and `sni`. This is a configuration-formatting error, not rejection of TLS server parameters. No production HAProxy config was changed or reloaded.

Correction: keep the full server directive on one logical line (or use explicit shell-generated single-line output), then validate the TLS backend syntax offline. After that, validate the exact `tcp-check connect/send/expect` syntax separately before introducing any health-check credential.

The corrected single-line server syntax was accepted by HAProxy, but the temporary test file contained only a backend and no frontend/listen section. `haproxy -c` therefore printed:

```
Configuration file has no error but will not start (no listener) => exit(2).
```

This is not a syntax failure. It explicitly confirms that HAProxy found no configuration error; exit code 2 is produced because the isolated test file has no listener and therefore would not start as a runtime configuration. No production config was changed or reloaded.

The corrected offline validation then passed on both ulc-01 and ulc-02 by adding a harmless dummy loopback frontend only inside the temporary test configuration. No listener was actually started because validation still used `haproxy -c` only.

Validated on both HAProxy nodes:

```
TLS backend server-line syntax: PASS
tcp-check TLS connect syntax: PASS
tcp-check send/expect syntax: PASS
combined three-backend TLS health-check grammar: PASS
production /etc/haproxy/haproxy.cfg remains valid: PASS
```

This proves HAProxy 2.8.16 accepts the required backend TLS, CA verification, SNI, tcp-check connect, tcp-check send, and tcp-check expect grammar. The production HAProxy configuration remains untouched and no reload/restart occurred.

Result:

```
Phase 8C.13 HAProxy offline TLS/tcp-check syntax validation: PASS
```

The dedicated least-privilege HAProxy health-check ACL identity was then deployed successfully on all three Valkey nodes.

Identity:

```
username = haproxy-health
password = separate randomly generated health-check credential
credential storage = /root/lorawan-secrets/valkey-haproxy-health.txt on ulc-03 during commissioning
```

The ACL is persisted symmetrically in /etc/valkey/valkey.conf on ulc-01, ulc-02, and ulc-03 and was also applied at runtime with ACL SETUSER, without restarting Valkey or changing replication topology.

Effective ACL policy:

```
on
-@all
+ping
+info
```

Observed validation on every node:

```
PING -> PASS
INFO replication -> PASS
GET -> NOPERM / denied
SET -> NOPERM / denied
```

Role detection through the dedicated account also matched the commissioned topology:

```
ulc-01 role=slave
ulc-02 role=slave
ulc-03 role=master
least-privilege role detection: PASS
```

Backups were created before each persistent ACL change. The main Valkey application/replication password was not reused for HAProxy health checks and no production HAProxy configuration was changed.

Result:

```
Phase 8C.13 dedicated HAProxy health-check user: PASS
```

The dedicated HAProxy health credential was then staged successfully on both HAProxy nodes under /etc/haproxy/secrets/valkey-health.env, with directory 0750 root:haproxy and file 0640 root:haproxy. The haproxy service account can read this dedicated credential and /etc/haproxy/ca/valkey-ca.crt, while /etc/lorawan-pki/valkey/server.key remains inaccessible. Secret equality between ulc-03's root-only source and both HAProxy copies was verified by SHA-256 comparison without printing the credential.

Observed checkpoint:

```
ulc-01 health credential access: PASS
ulc-01 CA access: PASS
ulc-01 Valkey private-key isolation: PASS
ulc-01 credential equality: PASS
ulc-02 health credential access: PASS
ulc-02 CA access: PASS
ulc-02 Valkey private-key isolation: PASS
ulc-02 credential equality: PASS
```

The direct authenticated role queries and offline HAProxy role-check validation then completed successfully from both HAProxy nodes. Using only the dedicated haproxy-health identity, both ulc-01 and ulc-02 independently observed the same Valkey topology:

```
ulc-01 role=slave
ulc-02 role=slave
ulc-03 role=master
```

The complete HAProxy 2.8 check grammar also validated offline on both nodes:

```
TLS to backend + CA verification
AUTH haproxy-health <dedicated credential>
expect +OK
INFO replication
expect role:master
```

The credential was supplied through VALKEY_HAPROXY_HEALTH_PASSWORD and was verified absent from the temporary HAProxy config text. The production /etc/haproxy/haproxy.cfg remained valid and unchanged, port 16379 remained free on both HAProxy nodes, and no reload/restart occurred.

Result:

```
Phase 8C.13 authenticated HAProxy role-check preflight: PASS
```

Production design decision: the :16379 frontend will remain TCP/TLS passthrough for application traffic, so client TLS terminates at the currently selected Valkey node and the shared certificate identity valkey.internal.lorawan.com continues to be verified end-to-end. Do not add ssl to the production backend server lines, because that would wrap an already TLS-speaking client stream in a second TLS layer. The health check itself will still use tcp-check connect ssl with CA verification and SNI. A systemd EnvironmentFile=/etc/haproxy/secrets/valkey-health.env drop-in is required so HAProxy can resolve % [env(VALKEY_HAPROXY_HEALTH_PASSWORD)] without storing the secret in haproxy.cfg. Because this environment must be present in the HAProxy master process, commission the two HAProxy nodes sequentially and restart each one only after validating its candidate config, leaving the other HAProxy node available during the brief restart.

The first production :16379 deployment was then attempted on ulc-01 only. Safety checks, backup creation, candidate generation, systemd EnvironmentFile integration, candidate validation, HAProxy restart, service state, master-process environment inheritance, production config validation, and the new 10.104.0.2:16379 listener all passed. Existing PostgreSQL (15432/15433) and MQTT (8883) listeners remained present.

The first functional test exposed a routing defect:

```
PING=PONG
routed role=slave
FAIL: ulc-01 HAProxy did not route to writable primary
```

So TLS passthrough itself is working, but the production health-check behavior is not yet excluding replicas. ulc-02 has NOT yet received the production :16379 configuration and must remain untouched until ulc-01 is corrected.

Current safe state:

```
ulc-01 :16379 listener = active
ulc-01 TLS passthrough = PASS
ulc-01 writable-primary selection = FAIL (routed to replica)
ulc-02 production :16379 deployment = NOT STARTED
```

The exact production block on ulc-01 was then inspected and matched the intended design: frontend `valkey_primary` binds `10.104.0.2:16379`; the backend uses option `tcp-check`; TLS health connections use SNI `valkey.internal.lorawan.com`; AUTH uses the dedicated environment-backed `haproxy-health` credential; INFO replication is followed by expect string `role:master`; and all three Valkey nodes are listed as checked backends. The production block therefore contains no obvious backend-order or plaintext/TLS configuration mistake.

A likely remaining explanation was HAProxy initial server state: checked servers may initially be available until failed health checks mark replicas DOWN. A delayed read-only re-test was therefore started after confirming the real Valkey roles remained `ulc-01=slave`, `ulc-02=slave`, `ulc-03=master` and waiting 15 seconds for health checks to settle.

The delayed test then terminated at the very first `:16379` role query before printing any routed role. Because the wrapper used `set -e` and the `valkey-cli` call was inside command substitution, any non-zero client result aborted the script before the HAProxy log section could run. This new evidence means the endpoint may have transitioned from the initial unsafe state (replica considered eligible) to a state with no eligible backend at all. That would point to a runtime health-check failure affecting every backend, including the real primary, rather than only an initial-state window. This must be diagnosed explicitly; do not assume `init-state fully-down` is the fix yet.

The first runtime diagnostic checkpoint confirmed that HAProxy itself remains healthy and the production listener still exists after the role checks have had time to settle:

```
haproxy=active
LISTEN 10.104.0.2:16379
```

The smaller runtime diagnostic then identified the actual health-check failure. `10.104.0.2:16379` remained listening and HAProxy remained active, but the client received `SSL_connect failed: unexpected eof while reading`. HAProxy logs showed all three Valkey backends becoming DOWN at exactly the same point: `Layer7 timeout ... at step 3 of tcp-check (expect string '+OK')`. After that HAProxy reported backend `'valkey_primary_backend'` has no server available, and later frontend traffic was logged against `<NOSRV>`. Therefore the failure occurs before INFO replication and before the `role:master` matcher; HAProxy is not receiving the `+OK` response to its AUTH command.

The production AUTH rule currently uses `tcp-check send-lf "AUTH haproxy-health %[env{VALKEY_HAPROXY_HEALTH_PASSWORD}]\r"`. HAProxy's documented Redis role-check examples use exact protocol sends ending in `\r\n`, such as `tcp-check send info\ replication\r\n`. The next safe proof is a temporary loopback-only HAProxy instance on ulc-01 using the same dedicated credential but rendering an exact static `tcp-check send AUTH\ haproxy-health\ <credential>\r\n` into a mode-0600 temporary config. This must not touch `/etc/haproxy/haproxy.cfg` or the systemd service. If that temporary check marks the replicas DOWN at `role:master` and ulc-03 UP, the root cause is the production `send-lf AUTH` framing rather than TLS, ACL permissions, or role detection. Destroy the temporary config/process after the test, keep ulc-02 production deployment untouched, and only then correct the production check.

A temporary loopback-only HAProxy proof was then run on ulc-01 using exact CRLF sends for both AUTH and INFO replication. The temporary configuration validated successfully, started successfully, and was protected mode 0600. The first functional request was issued only about five seconds after startup and still returned a replica (`role:slave`). This result does not yet disprove the exact-CRLF correction because the temporary backend used `timeout check 6s`: a replica can remain initially eligible while its first `role:master` expectation is still waiting to time

out. The proof therefore ended too early to distinguish framing failure from the initial health-check convergence window.

The settled exact-CRLF proof then passed. After 12 seconds, ulc-01 and ulc-02 were both marked DOWN specifically at step 5, expect string 'role:master', while ulc-03 remained eligible. Ten consecutive requests through the temporary loopback frontend all returned role:master, with master_routes=10, non_master_routes=0, and errors=0. This confirms the health-check protocol itself works when AUTH and INFO replication are sent with exact CRLF framing. It also confirms the initial unsafe replica routing was caused by the health-check convergence window: checked backends were initially eligible before their first role check completed.

Production correction definitely requires replacing the broken AUTH/INFO framing with exact CRLF and using a bounded role matcher. The isolated min-recv proof then passed on HAProxy 2.8.16: tcp-check expect min-recv 64 string role:master marked ulc-01 DOWN in 5 ms and ulc-02 DOWN in 14 ms with Layer7 invalid response, while ulc-03 remained eligible. After only a two-second wait, ten consecutive requests returned role:master, with master_routes=10, non_master_routes=0, and errors=0. This eliminates the earlier six-second convergence delay for replicas.

Validated HAProxy 2.8 production behavior is therefore: TLS health connection with SNI and CA verification, exact CRLF for AUTH and INFO replication, min-recv 64 on the role:master expectation, and the dedicated haproxy-health ACL identity. Do not use init-state fully-down on these 2.8 nodes. Apply the correction to ulc-01 only first, validate and reload HAProxy, wait briefly for checks to converge, and require repeated master-only routing before deploying the same corrected block to ulc-02.

The first ulc-01 production correction attempt did not reach validation or reload. Its line-aware Python rewrite succeeded in locating all three target rules, but it wrote the HAProxy escape sequences as doubled backslashes (\\r\\n) rather than the required single escaped CRLF tokens (\r\n). The immediately following literal assertions therefore failed and the wrapper exited before haproxy -c or systemctl reload haproxy. A second normalization pass corrected only those two send-lf lines, preserved min-recv 64, and passed exact-line assertions before reload.

The final ulc-01 production correction is now validated and live. The production rules are tcp-check send-lf "AUTH haproxy-health %[env(VALKEY_HAPROXY_HEALTH_PASSWORD)]\r\n", tcp-check send-lf "INFO replication\r\n", and tcp-check expect min-recv 64 string role:master. haproxy -c passed, HAProxy reloaded successfully, and the existing PostgreSQL 15432/15433, MQTT 8883, and Valkey 16379 listeners all remained present. Health logs marked ulc-01 and ulc-02 DOWN at the role:master matcher in 6 ms and 5 ms respectively, while ulc-03 remained eligible. Ten consecutive requests through 10.104.0.2:16379 all returned role:master (master_routes=10, non_master_routes=0, errors=0). A write through the ulc-01 HAProxy endpoint succeeded and the value was verified directly on the current ulc-03 primary, then the test key was removed. Therefore ulc-01 production writable-primary routing is PASS.

The same proven writable-primary configuration is now deployed on ulc-02 at 10.104.0.4:16379. The dedicated health credential and CA were already present and readable only by the HAProxy service as intended; the Valkey private key remained inaccessible. The candidate configuration passed exact AUTH/INFO CRLF assertions, min-recv 64, secret-exclusion, and haproxy -c validation. The systemd EnvironmentFile drop-in was installed, HAProxy restarted successfully so the master process inherited the health credential, and existing PostgreSQL 15432/15433 plus local Mosquitto 8884 listeners remained present. Health logs rejected ulc-01 and ulc-02 as replicas in 7 ms and 6 ms respectively. Ten consecutive requests through 10.104.0.4:16379 all returned role:master with zero errors. Both HAProxy endpoints then independently reported role:master; a write through ulc-02 was read successfully through ulc-01 and verified directly on ulc-03 before cleanup. Phase 8C.13 dual HAProxy writable-primary routing is PASS.

Phase 8C.14 completed the end-to-end automatic failover proof. The starting primary was ulc-03 (10.104.0.8) with healthy Sentinel quorum and both HAProxy :16379 endpoints routing to the writable master. A pre-failover key was written through ulc-01 HAProxy and read through ulc-02. Only the ulc-03 Valkey service was then stopped; its Sentinel remained active. Sentinel elected ulc-02 (10.104.0.4) as the new primary after seven polling attempts,

and all three Sentinels agreed on that new primary. Direct verification showed ulc-02 as `role:master` with ulc-01 online as a replica.

Both HAProxy endpoints automatically followed the promoted primary with no HAProxy configuration change or reload. Each endpoint was briefly unavailable for one polling attempt during convergence, then routed to `role:master`; ten consecutive post-failover requests through each endpoint were master-only with zero errors. The pre-failover value survived, a new write through ulc-02 HAProxy succeeded, the new value was read through ulc-01 HAProxy, and it was verified directly on the new ulc-02 primary. ulc-03 Valkey was then restarted; it transitioned from its prior master state to `role:slave`, followed ulc-02, and reached `master_link_status:up` on attempt 9. Final topology was ulc-02 primary with ulc-01 and ulc-03 online as replicas (`connected_slaves:2`). Sentinel quorum remained healthy, the test key was removed, and no manual failback was performed.

Phase 8C Valkey HA is therefore complete: TLS-only Valkey on three nodes, authenticated replication, three TLS Sentinels with quorum 2, automatic promotion and replica recovery, dual HAProxy writable-primary endpoints on 10.104.0.2:16379 and 10.104.0.4:16379, verified TLS health checks using the least-privilege haproxy-health identity, exact CRLF AUTH/INFO replication framing, `min-recv 64` master-role rejection, master-only routing, and successful automatic HAProxy convergence after Sentinel failover. Current primary is ulc-02 (10.104.0.4). Keep the Sentinel-elected primary in place; do not manually fail back.

Acceptance criteria for the completed Valkey HA layer:

```
Three Valkey nodes installed
One primary
Two replicas
Three Sentinel members
Automatic promotion tested
HAProxy follows writable primary
```

Deployment documentation rule

All executed commands, configuration decisions, validation output, failures, and corrections must be recorded in this MDS before repository commit.

The operator performs the final Git commit and push after review.

Next: [09-chirpstack-cloud-cluster.md](#).

Current gateway-facing MQTT authentication correction - 2026-08-28

The target design earlier in this manual shows the gateway-facing Mosquitto backend :8884 with `require_certificate true` and `use_identity_as_username true`. Later live inventory is authoritative for the current runtime: both brokers presently expose :8884 with `require_certificate=false`, `allow_anonymous=false`, no `use_identity_as_username`, and no gateway-specific ACL/plugin. Treat the earlier mTLS block as the **target configuration**, not proof that gateway client-certificate authentication is already commissioned.

The cloud gateway mTLS boundary is therefore still open even though the broker TLS/failover foundation is healthy. Before public `mqtt.<DOMAIN>:8883` is activated, commission the gateway authentication boundary canary-first on ulc-01 then ulc-02: issue one cloud MQTT `clientAuth` certificate whose CN is the authoritative Gateway EUI `0016c001f139a1cb`, add an exact-EUI ACL for the `as923/gateway/0016c001f139a1cb/... event/state/command` hierarchy, enable client-certificate authentication on the gateway-facing :8884 listener, and prove positive mTLS plus cross-EUI/no-certificate denial through the existing anchor :8883 TLS-passthrough path. This work does not require the final public FQDN or Reserved IPv4 and should be completed before provider public activation.

Gateway MQTT cloud client certificate issuance PASS - 2026-08-28

The real physical Gateway EUI 0016c001f139a1cb now has a dedicated cloud MQTT client identity issued on ulc-03 from the commissioned internal CA. Issuance directory: /root/lorawan-pg-ca/gateway-0016c001f139a1cb-issuance-20260828T015149Z; protected transfer bundle is its transfer/ subdirectory. The certificate subject is CN = 0016c001f139a1cb, issuer CN = LoRaWAN PostgreSQL Internal CA, serial D8732205912F3C3AC56E0A01E1E10583, validity 2026-08-28 01:51:51Z through 2027-09-29 01:51:51Z, SHA-256 fingerprint 82:C6:9A:D7:12:5D:8C:45:F3:8F:BA:AB:F9:6E:7E:3B:41:F4:BE:78:95:FE:16:05:FD:58:2E:09:9D:F3:0A:ED, and certificate SHA-256 f348cef6e280dff82722ff908cc96e694faf183eec1dffc55a7abc690cb472d8.

The certificate chains successfully for sslclient, is rejected for sslserver, and its public key matches the RSA-3072 private key. The issuing CA SHA-256 remains 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb; the existing CA serial-file SHA-256 remained byte-identical at 50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d because issuance used an explicit random serial. GATEWAY_MQTT_CLIENT_CERT_ISSUANCE=PASS. No Mosquitto, HAProxy, DNS, firewall, or gateway runtime was changed by issuance.

Next boundary: canary-harden ulc-01 gateway-facing Mosquitto :8884 only to require client certificates, map certificate CN to MQTT username, and enforce an exact-EUI as923 gateway ACL. Preserve ChirpStack :8885 and Node-RED :8886. After ulc-01 passes direct mTLS verification with the new gateway certificate, repeat the same bounded rollout on ulc-02.

ulc-01 gateway mTLS canary PASS - 2026-08-28

ulc-01:8884 was hardened from server-TLS-only to gateway client-certificate authentication without changing the ChirpStack :8885 listener. The active gateway listener now has require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. The exact Gateway EUI 0016c001f139a1cb is permitted to write only its own as923/.../event/# and state/# topics and read only its own command/# hierarchy. Mosquitto restarted successfully, :8884 and :8885 remained listening, the issued gateway certificate completed TLS and MQTT CONNECT/SUBSCRIBE successfully, and a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015613Z. Next boundary: apply the same bounded change to ulc-02 and prove the same certificate/ACL behavior there before transferring the cloud certificate bundle to Gateway OS.

ulc-02 gateway mTLS rollout PASS - 2026-08-28

ulc-02:8884 now matches the proven ulc-01 gateway-authentication boundary: require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. Gateway EUI 0016c001f139a1cb is limited to its own AS923 event/state/command hierarchy. Mosquitto restarted successfully; :8884 and ChirpStack :8885 remained listening. The issued gateway certificate completed TLS, MQTT CONNECT, and own-command SUBSCRIBE successfully, while a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015801Z. Therefore both cloud MQTT broker backends now enforce the intended per-gateway mTLS identity. Next boundary: transfer only the three-file cloud certificate bundle (ca.crt, 0016c001f139a1cb.crt, 0016c001f139a1cb.key) to Gateway OS; do not change the working local Mosquitto topology yet.

ulc-02 gateway mTLS state observed - 2026-08-28

Live read-only inspection after the first rollout wrapper showed ulc-02 is already in the intended gateway mTLS state: /etc/mosquitto/conf.d/tls.conf has listener 8884, require_certificate true, allow_anonymous false, use_identity_as_username true, and acl_file /etc/mosquitto/gateway.acl; /etc/mosquitto/gateway.acl exists as 0640 root:mosquitto; per_listener_settings true remains active; and the dedicated ChirpStack listener remains 10.104.0.4:8885 with its own password/ACL files. Therefore do not reapply the mutation. The remaining boundary is a direct positive mTLS/MQTT proof with gateway certificate CN 0016c001f139a1cb plus a no-client-certificate rejection proof.

Both cloud gateway MQTT brokers mTLS acceptance PASS - 2026-08-28

Direct verification from ulc-03 against ulc-02:8884 using the issued gateway identity CN = 0016c001f139a1cb passed TLS, MQTT CONNECT, and subscription to the gateway's own as923/gateway/0016c001f139a1cb/command/# hierarchy. A client without a certificate was rejected. Combined with the earlier ulc-01 canary proof, both gateway-facing Mosquitto backends now enforce per-gateway mTLS and the exact EUI ACL while preserving the dedicated ChirpStack :8885 listeners. The cloud broker-side gateway authentication boundary is therefore complete. Next: transfer only ca.crt, 0016c001f139a1cb.crt, and 0016c001f139a1cb.key from the protected ulc-03 issuance bundle to Gateway OS; do not alter the local loopback broker topology during certificate import.

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Reproducible Cloud Evidence Deployment Bundle

This directory is the tracked and **commissioned** server-side deployment bundle for the four Go evidence/attestation services: ingest, MQTT collector, verifier, and Fabric adapter. The base profile is live on the three cloud hosts. After the governed ULC-01 qualification, ULC-01 now runs the explicit enabled overlay as the continuous Fabric writer; ULC-02 remains intentionally disabled standby until the HA fencing gate passes. Rebuilding a server should require the repository plus protected credentials/PKI and the selected durable S3-compatible service; it should not require reconstructing Compose commands from chat history.

The Fabric adapter has two deliberately separate deployment states. The base `compose.yml` runs `adapter-1/2` with `FABRIC_ADAPTER_ENABLED=false`, so they expose only local health/readiness and do not open PostgreSQL, read an OpenBao SecretID, or contact Fabric. Actual ledger submission requires the explicit `compose.fabric-adapter-enabled.yml` overlay and must pass `fabric-adapter-enable-preflight.sh` first.

Frozen HA placement

```
ulc-01 / 10.104.0.2
  ingest-1
  collector-1
  fabric-adapter-1 enabled continuous writer -> HRC Fabric through the governed private Gateway

ulc-02 / 10.104.0.4
  ingest-2
  verifier-1
  fabric-adapter-2 disabled standby -> do not enable before HA fencing acceptance

ulc-03 / 10.104.0.8
  collector-2
  verifier-2
  no Fabric adapter worker
```

Every runtime starts with the documented ceiling 192 MiB / 0.20 CPU. That is a measurement guardrail, not permission to remove HA if the 2-GiB hosts are too small.

Files

<code>compose.yml</code>	base four-service definition; adapter is hard-frozen disabled
<code>compose.collector-mtls.yml</code>	collector client-certificate mounts
<code>compose.fabric-adapter-enabled.yml</code>	explicit activation overlay; never use for standby staging
<code>release.env.example</code>	template for immutable OCI digest references
<code>release.commissioned.env</code>	exact non-secret commissioned four-image GHCR release
<code>hosts/*.env.example</code>	exact node placement + host ports + enabled-adapter mount paths
<code>env/common.env.example</code>	non-secret common DB/S3 settings
<code>env/ingest.env.example</code>	protected ingest DB/S3 credentials + TLS paths

```

env/collector-password.env.example    legacy/reference input only; production preflight rejects password mode
env/collector-mtls.env.example        protected collector DB/S3 + client-certificate paths
env/verifier.env.example              protected verifier DB/S3 settings
env/fabric-adapter.env.example        protected DB/Fabric topology settings used only after handoff
host-preflight.sh                    early read-only live host/capacity/service/port gate
preflight.sh                         final base/standby staged-config gate
fabric-adapter-enable-preflight.sh    separate no-mutation gate before enabling ledger submission

```

The Compose definition runs every container as numeric 65532:65532, read-only root filesystem, all Linux capabilities dropped, no-new-privileges, PID limit 128, bounded Docker logs, and no in-image shell/health utility. Collector/verifier/Fabric-adapter health listeners bind host loopback only. Ingest binds only the selected private host IP/backend port. The public Evidence route is now commissioned through the shared HAProxy SNI frontend at smartagri-evidence.duckdns.org:443.

Inputs that intentionally remain external

A real base deployment must supply:

1. four OCI image **digest** references built from ../packaging/;
2. the selected SeaweedFS 4.41 raw-evidence cluster from seaweedfs/, commissioned with 010 cross-rack placement and the local internal-TLS create-only S3 frontend;
3. per-role PostgreSQL login credentials, each a member only of its corresponding NOLOGIN group role;
4. per-role S3 credentials with minimum object permissions;
5. Evidence server/client PKI and the commissioned PostgreSQL/MQTT/S3 CA files;
6. dedicated read-only MQTT collector broker identities;
7. free private/loopback host ports confirmed on the live server.

Fabric activation uses the commissioned HRC Task 37 handoff. The adapter contract is CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion) with the exact immutable finalized_payload bytes supplied only as transient hrc.exact_payload, followed by QuerySourceBoundAnchor(SourceRecordID) and VerifySourceBoundDigest(SourceRecordID, observedDigest). The frozen LoRaWAN source identity is FABRIC_AUTHENTICATED_SOURCE_SYSTEM_ID=lorawan-gateway-evidence; the private Gateway is 10.104.0.7:7051 with TLS name peer1.hrc.local, and ULC-01 has the commissioned dedicated non-admin adapter identity. ULC-02 remains a protected write-disabled standby until its HA fencing/ownership gate is accepted. The old 10.43.25.198:7051 K3s ClusterIP and legacy CreateAnchor, QueryAnchor, QueryAnchorByRecordID, and VerifyDigest APIs must not be used.

The repository does not guess image digests, secrets, or unused evidence-service host ports. The raw-store endpoint itself is now frozen to https://evidence-objects.internal.lorawan.com:18443; each evidence container resolves that name to its local host VPC IP, while SeaweedFS's raw S3 listener remains loopback-only.

Fixed host layout

Use these directories on every server:

```

/etc/lorawan-cloud/gateway-evidence/
  release.env
  host.env
  common.env
  ingest.env      # only on ingest hosts
  collector.env   # only on collector hosts
  verifier.env    # only on verifier hosts
  fabric-adapter.env # ulc-01 active writer / ulc-02 protected disabled standby

```

```

/etc/lorawan-pki/gateway-evidence/
  postgres-ca.crt
  s3-ca.crt
  ingest-server.crt / ingest-server.key / gateway-client-ca.crt # ingest hosts
  mqtt-ca.crt                                           # collector hosts
  collector-broker*.crt/key                             # mTLS collector mode only

# ulc-01/02 only, created only at Fabric activation time:
/etc/lorawan-cloud/fabric-adapter/
  role_id
  secret_id

/etc/lorawan-pki/fabric/
  tls-ca.crt
  client.crt
  client.key

```

Protected role env files must be `root:root 0600`. Runtime private keys are bind-mounted into a numeric non-root container; install them as `root:<gid 65532> 0440`. Public certificates/CA files must also be readable by runtime `65532:65532`: use either `root:65532 0440` or a root-owned non-writable world-readable public file such as `0444`. The preflight checks actual mode/GID readability so a `0600 root:root` CA cannot pass and then break container startup.

If GID 65532 is unused on the server, a deployment operator may create a named host group for clarity before installing keys:

```
sudo groupadd --system --gid 65532 evidence-runtime
```

Do not change an existing GID 65532 owner blindly. If that numeric GID is already allocated, stop and review the container/runtime ownership design instead of weakening key permissions.

Prepare configuration

Copy examples, then replace placeholders **outside Git**:

```

sudo install -d -m 0750 -o root -g root /etc/lorawan-cloud/gateway-evidence
sudo install -d -m 0750 -o root -g root /etc/lorawan-pki/gateway-evidence

# Example only: choose the matching host file.
sudo install -m 0644 -o root -g root release.env /etc/lorawan-cloud/gateway-evidence/release.env
sudo install -m 0644 -o root -g root host.env /etc/lorawan-cloud/gateway-evidence/host.env
sudo install -m 0644 -o root -g root common.env /etc/lorawan-cloud/gateway-evidence/common.env
sudo install -m 0600 -o root -g root ingest.env /etc/lorawan-cloud/gateway-evidence/ingest.env

```

Do not copy literal example placeholders into production.

Database credentials

`001_gateway_evidence.sql` creates three NOLOGIN group roles:

```

gateway_evidence_ingestor
gateway_evidence_collector
gateway_evidence_verifier

```

The production LOGIN identities are now frozen and live. Preserve these mappings rather than creating new ad-hoc names:

```

evidence_ingest_ulc01    -> gateway_evidence_ingestor
evidence_ingest_ulc02    -> gateway_evidence_ingestor

```

```
evidence_collector_ulc01 -> gateway_evidence_collector
evidence_collector_ulc03 -> gateway_evidence_collector
evidence_verifier_ulc02  -> gateway_evidence_verifier
evidence_verifier_ulc03  -> gateway_evidence_verifier
```

All six are SCRAM LOGIN roles with exactly one intended NOLOGIN authority membership and inherited CONNECT on `lorawan_telemetry`. Protected plaintext custody remains outside Git/Markdown under the commissioned root-only bootstrap boundary.

Every DSN uses the commissioned logical endpoint `pgbouncer.internal.lorawan.com:6432`, database `lorawan_telemetry`, hostname-verified TLS, and the mounted PostgreSQL CA. Because this internal name is not assumed to exist in public DNS, Compose maps it inside every container to the **local node private IP** (`10.104.0.2`, `.4`, or `.8`), preserving the local PgBouncer -> local HAProxy -> Patroni-primary path. The Go pool also enforces SCRAM-SHA-256 and verifies role membership/writable-primary routing on every new physical DB session.

All six PostgreSQL LOGIN roles and the three-node PgBouncer expansion are commissioned. Every PgBouncer node has the exact ten-entry/six-evidence SCRAM userlist SHA-256 `665f1592c96ca276681a454b9cbcd6ab8ab0cbfb4594b8ddd443239db58df391`, with the original four-role verifier set preserved for rollback and proven byte-identical across the expansion. Fresh strict verify-full evidence authentication passed through all three physical `:6432` endpoints and routed to the current writable Patroni primary. Do not regenerate PgBouncer userlists again unless a later credential change requires it.

S3 credentials

The production object-store backend is selected and commissioned through S9: SeaweedFS 4.41 behind the internal HAProxy TLS/create-only endpoint `https://evidence-objects.internal.lorawan.com:18443`. The locked production helper write and cross-host retained-object verification are PASS. Runtime S3 identities are already installed with the following permission split:

```
ingest:    HeadBucket + GetObject + PutObject on evidence prefix
collector: HeadBucket + GetObject + PutObject on evidence prefix
verifier:  HeadBucket + GetObject only on evidence prefix
```

The application uses conditional create-if-absent writes. Do not grant object delete/overwrite administration to these runtime credentials.

Collector broker authentication

Production collector authentication is frozen to **mTLS** because the current commissioned `ulc-01:8884` and `ulc-02:8884` gateway-facing broker listeners require client certificates and map certificate identity to the MQTT username. Do not reopen password mode merely for convenience.

```
collector-mtls.env.example -> /etc/lorawan-cloud/gateway-evidence/collector.env
EVIDENCE_COLLECTOR_AUTH_MODE=mtls
```

Issue dedicated collector `clientAuth` identities whose Mosquitto usernames receive only `read` `as923/gateway/+/#/event/#`; grant no publish or command permission. `compose.collector-mtls.yml` is supplied automatically by `preflight.sh` and must also be supplied to the later activation command. The password example remains reference material only and is rejected by production `preflight`.

Each collector replica has **two distinct client IDs**, one per fixed backend (`10.104.0.2:8884` and `10.104.0.4:8884`). Across two collectors that produces four independent persistent read-only sessions.

Fast live-host preflight

Before creating evidence credentials, applying the migration, or choosing final ports, run the same observation-only check from an **already-authorized SSH session** on each server:

```
sudo bash host-preflight.sh
```

Run it on `ulc-01`, then `ulc-02`, then `ulc-03`. It verifies current private-IP identity, memory/disk headroom, UID/GID 65532 collision risk, Docker/Compose/Buildx availability, container RSS, HAProxy/PgBouncer state, Patroni/etcd visibility, current :8884 mTLS policy where Mosquitto lives, the existing anchor :443, and dynamically suggests currently free ingest/health ports. Suggested ports are observations only; they are not reserved and should be copied into protected host configuration immediately before the final staged-config preflight.

The script deliberately does not require the repository runner to possess server private keys. Never copy a workstation SSH private key into this repository or onto a Droplet merely to automate this check.

Mandatory final staged-config preflight

After files are installed, but **before pulling or starting anything**:

```
sudo ./preflight.sh \  
  /etc/lorawan-cloud/gateway-evidence/release.env \  
  /etc/lorawan-cloud/gateway-evidence/host.env
```

It verifies:

- exact frozen node/profile placement, including adapter-1 on `ulc-01` and adapter-2 on `ulc-02` only;
- correct private IP on the host;
- all four image references are immutable `@sha256` references;
- the base Fabric adapter profile remains standby-only and accepts no DB/OpenBao/Fabric credential mounts;
- selected host ports are numeric and currently free;
- protected env/key ownership and permissions;
- required CA/certificate files are actually readable by runtime 65532:65532 and private keys are root:65532 0440;
- `pgbouncer.internal.lorawan.com` is mapped inside containers to the local node private IP;
- production backend is `s3`, never the dev filesystem;
- collector auth-mode-specific inputs;
- PostgreSQL DSNs use the logical PgBouncer endpoint + `verify-full`;
- `docker compose config --quiet` succeeds.

Expected final marker:

```
EVIDENCE_DEPLOYMENT_PREFLIGHT=PASS
```

The script does not pull images, start containers, issue credentials, alter permissions, or mutate the database.

Base/standby activation command

This boundary was commissioned on 2026-09-01 after image digests, SeaweedFS S9, the DB/PgBouncer roles, Evidence PKI, shared-443 design, and all three host preflights passed. The command remains the reproducible start/recovery path. Production collectors use mTLS, so collector hosts include the collector override:

```
sudo docker compose \
  --env-file /etc/lorawan-cloud/gateway-evidence/release.env \
  --env-file /etc/lorawan-cloud/gateway-evidence/host.env \
  -f compose.yml \
  -f compose.collector-mtls.yml \
  up -d
```

On ulc-02 there is no collector profile, so use only `compose.yml`. The Fabric adapter profile is present on ulc-01/02 but remains disabled by the base Compose definition. Start and verify one logical boundary at a time; do not bring all six evidence replicas plus two adapter standbys up blindly.

Fabric adapter activation is a separate gate

Do not issue/install an OpenBao SecretID merely because the standby container is healthy. After the external Fabric handoff is complete, install the protected adapter env/identity files, then run:

```
sudo ./fabric-adapter-enable-preflight.sh \
  /etc/lorawan-cloud/gateway-evidence/release.env \
  /etc/lorawan-cloud/gateway-evidence/host.env
```

This preflight verifies the exact adapter host/worker identity, protected file modes, `fabric_adapter` PgBouncer TLS DSN, fixed OpenBao service path, Fabric certificate/private-key match, stable OpenBao TLS/health, external Fabric Gateway server TLS, and the combined Compose configuration. It performs no pull/start/credential issuance or state mutation.

Only after that marker is `FABRIC_ADAPTER_ENABLE_PREFLIGHT=PASS` may the explicit overlay be used for the one adapter being activated:

```
sudo docker compose \
  --env-file /etc/lorawan-cloud/gateway-evidence/release.env \
  --env-file /etc/lorawan-cloud/gateway-evidence/host.env \
  -f compose.yml \
  -f compose.fabric-adapter-enabled.yml \
  up -d fabric-adapter
```

On ulc-01 include `-f compose.collector-mtls.yml` as well when managing the whole host stack. If the external Fabric Gateway requires a separate **transport mTLS** client certificate rather than server-auth TLS plus the Fabric application identity currently implemented, stop at the preflight failure and extend/rebuild the adapter; never disable TLS verification to force the connection.

Commissioned immutable release - 2026-09-01

Tracked `release.commissioned.env` and the production release files use these immutable references. The current hosts may use protected exact preloaded-image overrides with `--pull never` because the legacy host registry credential cannot read the new private GHCR packages; do not weaken package privacy as a workaround. A rebuild should either provision authorized registry-read access or preload images whose content identity has been verified against the commissioned release.

```
EVIDENCE_INGEST_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-evidence-
ingest@sha256:f2fd695e8d2505a12202b4fabd92d035c562e16d4e308ccf9436a08a36aaec1a
EVIDENCE_COLLECTOR_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-mqtt-evidence-
collector@sha256:0bcaaa127f913157596a87077235d4f0717fa1c29f08d8aca8568c3167ed2cff
EVIDENCE_VERIFIER_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-evidence-
verifier@sha256:3416496fa563154fbd7209a575acd040b735b3c4f4159902fc886e78167195c0
EVIDENCE_ADAPTER_IMAGE=ghcr.io/jervis-umtc/lorawan/gateway-fabric-
adapter@sha256:cd4308e8985d74ea7fab957a2a4adadd1831b776a8f9af2fdd6291179df83e7a
```

All three authoritative base `preflight.sh` runs passed. Current placement is `ulc-01=ingest-1+collector-1+adapter-1-enabled`, `ulc-02=ingest-2+verifier-1+adapter-2-disabled`, `ulc-03=collector-2+verifier-2`. The base Compose remains deliberately fail-closed; ULC-01 reaches the enabled state only through the governed activation overlay and `preflight`. Every runtime uses numeric `65532:65532`, read-only rootfs, `cap_drop: ALL`, `no-new-privileges`, `pids_limit: 128`, `192 MiB`, `0.20 CPU`, bounded logs, and `restart: unless-stopped`.

SeaweedFS S9, Evidence PKI, four distinct read-only collector mTLS identities/ACLs, the dual-broker collector readiness gate, replicated ingest/verifier readiness, and the private/shared-443 normal path are PASS. ULC-01 Adapter-1 is the governed continuous Fabric writer with `FABRIC_ADAPTER_ENABLED=true`; ULC-02 Adapter-2 remains healthy `FABRIC_ADAPTER_ENABLED=false` standby until the HA fencing gate passes.

What is still not claimed

The real Gateway-01 / assembled EMU-01 v2 physical evidence lineage is PASS through Concentrator, MQTT, journal/uploader, Evidence ingest, checkpoints/segments, and trusted-decoder verification. On the final 2026-09-02 shutdown check, Gateway-01 had closed and receipted segment 28, its open journal was empty, and the cloud held 492 verified v2 events with 0 pending, 0 unseen, and 0 `evidence_gap / integrity_failure` rows. Both verifier replicas run the corrected continuous-discovery build that excludes already-seen v2 keys before applying the 100-row batch limit, preventing the old first-100 starvation condition. Public ChirpStack/Evidence/MQTT normal paths are also PASS. Remaining external claims are deliberately narrower: Reserved-IP reassignment/failover authority and controlled acceptance, plus the separate ULC-02 Fabric HA fencing/dual-worker acceptance gate. Do not redo the already-passed cloud evidence commissioning without a relevant state change.

Supporting runbook: 02-fabric-network-handoff.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

2. Fabric Network Handoff Package

The Fabric team creates and operates the network. Use this procedure to exchange the concrete values the adapter needs and to prove them in staging. Sensitive identity material must use the organization's protected provisioning channel rather than a normal message or issue.

HRC handoff status — 2026-09-08: ULC-01 EXTERNAL QUALIFICATION PASS. hrc-evidence version 1.1 / sequence 2 is live-qualified across all three Fabric peers. The Adapter uses the Task 37 source-bound API: `CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion)` with exact payload bytes only in transient key `hrc.exact_payload`, then `QuerySourceBoundAnchor(SourceRecordID)` and `VerifySourceBoundDigest(SourceRecordID, observedDigest)`. The legacy `CreateAnchor`, `QueryAnchor`, `QueryAnchorByRecordID`, and `VerifyDigest` functions are forbidden for Adapter identities. The frozen source identity is `lorawan-gateway-evidence`. ULC-01 (10.104.0.2) is commissioned against the restricted TLS-preserving Gateway 10.104.0.7:7051 with TLS name `peer1.hrc.local`, MSP `HrcMSP`, channel `hrc-channel`, chaincode `hrc-evidence`, and dedicated non-admin client identity `lorawan-gateway-evidence-ulc-01`; authoritative activation-preflight SHA-256 remains `d2e7c9a59270d248ea14b4bd035c671e1a9768ddb1ec8bfa527d675c3b3cca97`. The authorized one-record qualification for outbox 1960 / `SourceRecordID fdadcf14-741e-4293-ad38-39ba89369ea7` completed successfully with exact-payload SHA-256 `827e443857118c0f922c2f03e66f950a79092c74d8d8b947d489ace43d157b51`, Fabric transaction ID `dd11910c1e7c28edd11334a03cce10d80d574692733458c9059d2775be5131b8`, and HRC record ID `hrc-record-ad1258ae5ee243c01d1e091c684df1c950589bcbcd820d736a257f8cd052bfb`. The qualification runner only marks the row `confirmed` after authoritative commit success, `QuerySourceBoundAnchor` returns matching source/record/digest/payload-length fields, and `VerifySourceBoundDigest` returns `MATCH`; final outbox state is `confirmed`, `attempts=1`, with prepared transaction and `commit-status` request retained. The one-shot qualification container is gone. As re-verified on 2026-09-09, the ULC-01 continuous Adapter is active with `FABRIC_ADAPTER_ENABLED=true`; the production activation preflight passes and 325 records have reached `confirmed` after the worker start. ULC-02 (10.104.0.4) remains `FABRIC_ADAPTER_ENABLED=false/deferred`; enabling its HA worker is a separate later boundary that requires the lease-renewal/ownership-fencing gate and must not trigger a repeat of candidate 1960.

2.1 Information the Fabric team must provide

Request the following:

Item	Required value	Why it is needed
Fabric version	Exact network and peer versions	Client compatibility and support

Item	Required value	Why it is needed
Organization name	For example, AgricultureOrg or PortOperatorOrg	MSP and policy identity
MSP ID	HrcMSP	Transaction signing and authorization
Channel name	hrc-channel	Ledger routing
Chaincode name	hrc-evidence	Contract lookup
Chaincode version and sequence	Current definition (still request for change coordination)	Upgrade coordination
Contract namespace	empty/default; not Caliper alias hrcEvidence	Fabric Contract API lookup
Required transaction	CreateSourceBoundAnchor(SourceRecordID, sourceType, producer, producedAt, schemaVersion) plus transient hrc.exact_payload	Task 37 application contract
Query transaction	QuerySourceBoundAnchor(SourceRecordID)	Verification
Digest verification transaction	VerifySourceBoundDigest(SourceRecordID, observedDigest) and require MATCH	Post-commit verification
Endorsement policy	Organizations required to endorse	Retry and availability planning
Fabric Gateway endpoint	Host and port	Client connection
Peer endpoint	If the client profile requires it	Gateway and TLS routing
TLS CA certificate	Out-of-band trusted certificate	Prevent endpoint impersonation
Client identity certificate	Public certificate for the adapter	Authentication
Client private key	Securely provisioned, never emailed in plain text	Transaction signing
Connection profile	Network connection details	Client configuration
Event behavior	Commit and chaincode event format	Downstream processing
Rate limits	Transactions per second and burst limits	Back-pressure design
Maintenance windows	Expected outages and upgrade process	Operational planning

Fabric channels define membership and ledger separation, while policies determine who can act. Do not invent channel names, MSP IDs, or endorsement rules in the Node-RED flow.

2.2 Information your team must provide

Give the Fabric team:

- Gateway EUI: <GATEWAY_EUI> obtained from the active Concentrator configuration;
- LoRaWAN region: <CONFIRMED_REGION_ID> with the site authorization evidence;
- ChirpStack application and device identifier conventions;
- DevEUI normalization rule;
- the frozen EMU-01 payload-v2 decoded field dictionary, including units and sensor_validity_bitmap semantics;

- MQTT topic pattern: application/+device/+event/up;
- TimescaleDB table and view names;
- timestamp policy: UTC;
- duplicate key policy;
- expected event volume and aggregation window;
- example sanitized uplink JSON;
- example canonical attestation JSON;
- desired query and verification operations;
- data classification for each field;
- AuthenticatedSourceSystemID: lorawan-gateway-evidence;
- DigitalOcean VPC: 10.104.0.0/20;
- ULC-01 private/public source addresses: 10.104.0.2 / 143.198.205.54;
- ULC-02 private/public source addresses: 10.104.0.4 / 165.22.253.127.

Do not send passwords, private keys, LoRaWAN root keys, connection profiles containing secrets, or raw personal data in a normal project chat or issue. Exchange certificates and private keys through the approved secure provisioning channel and record only their identifiers, owners, expiry dates, and storage locations in the handoff.

2.3 Build the handoff request

Use the template below to collect the environment-specific values. Each field is consumed by the adapter configuration, TLS validation, contract invocation, capacity plan, or support procedure; omit fields that do not apply rather than inventing values:

```
Integration name: LoRaWAN telemetry attestation
Environment: pilot / staging / production
Submitting organization:
MSP ID:
Channel:
Chaincode:
Chaincode version:
Fabric Gateway endpoint:
TLS server name:
Required transaction:
Required query:
Required event:
Client identity name:
Endorsement policy:
Expected transaction rate:
Maximum accepted delay:
Private data requirement:
Test window:
Support contact:
Certificate rotation contact:
```

2.4 Test the handoff

The handoff is usable only when the Fabric team has supplied and the application team has verified:

- a reachable staging Gateway endpoint;
- a valid test identity;
- the trusted TLS CA material;
- the channel and chaincode names;
- a transaction that can be evaluated;

- a transaction that can be submitted;
- a documented commit-status result;
- a documented failure response for duplicate events;
- a certificate rotation procedure;
- a support and incident path.

Do not treat a file called `connection profile` as sufficient by itself. The adapter also needs a verified endpoint, server-name expectation, trusted TLS roots, dedicated client identity, protected private key, exact channel and chaincode contract, authorization policy, commit-status behavior, and revocation process.

2.5 Security boundary

The integration application should connect to a Fabric Gateway that belongs to the same organization as its client identity. Store the private key in a protected secret mechanism or file with restricted permissions. Never store it in Node-RED flow JSON, a dashboard, a Git repository, or a debug message.

References:

- [Fabric Gateway application model](#)
- [Certificates and identity management](#)
- [Fabric channels](#)

Next: [03-integration-architecture.md](#)

Canonical source: `deployment/technology-guides/17-pki-tls-and-service-identity.md`. For any mutable production-status statement, check the dated baseline and verified live state.